

# GEF Secure — Informe técnico de implementación

## (Fases 1 a 4 + mejoras posteriores)

### Documento de referencia para la redacción del informe de tesis.

Describe, con nivel de detalle técnico, lo efectivamente implementado y verificado sobre el sistema GEF Secure a partir del `Plan_Unificado_Implementacion_GEF_Secure.md`. No reemplaza ese plan (que era prospectivo) sino que documenta el resultado real: qué se construyó, qué decisiones de diseño se tomaron durante la implementación, qué defectos se encontraron y corrigieron, y cómo se verificó cada fase antes de darla por cerrada.

**Alcance:** Fase 1 (Catálogo de software), Fase 2 (Jerarquía Activo/Componente), Fase 3 (Autenticación) y Fase 4 (RBAC + Scope) — cuerpo principal del documento (§1 a §10). A eso se suma una **§11 separada** con las mejoras implementadas después de cerrar la Fase 4, ya sobre el sistema con roles y scope funcionando: rediseño del Centro de Escaneos, unificación de Inventario y Activos, borrado lógico de activos, correlación de vulnerabilidades por escaneo histórico, rediseño de paneles de detalle como popup, unificación de idioma en la prioridad de vulnerabilidades, trazabilidad completa de escaneos (relación real Scan ↔ hallazgo ↔ vulnerabilidad persistente, con ciclo de vida NEW/PERSISTING/REOPENED/RESOLVED, y comparación entre dos escaneos), catálogos administrables en Config (Entornos, Tipos de Host, Ecosistemas), una auditoría de datos "congelados vs. en vivo" que corrigió tres casos de valores duplicados que podían desincronizarse de su fuente real, la corrección de raíz de la correlación de escaneos con GitHub Advisory (filtrado por inventario real, en vez de un lote genérico sin filtrar) — verificada con hallazgos reales en los cuatro tipos de escaneo y en el disparador programado diario —, la corrección del detalle de un escaneo puntual para que filtre por su `scan_id` real en vez de una ventana de tiempo que podía mezclar resultados de escaneos distintos, y drill-down por activo dentro del detalle de un escaneo GLOBAL/ENTORNO. Todo el trabajo descrito está implementado, verificado mediante pruebas automatizadas y en vivo, y el proyecto quedó en estado funcional de punta a punta.

## 1. Resumen ejecutivo

GEF Secure partía de tres brechas estructurales, diagnosticadas por separado y unificadas en un solo plan: (1) el software instalado se identificaba con texto libre, sin correlacionar contra una fuente de vulnerabilidades real; (2) no existía distinción entre "activo real" (servidor, aplicación) y "componente de software" (paquete instalado); (3) el sistema no tenía autenticación de ningún tipo, por lo que el campo `role` de usuario no tenía ningún efecto práctico.

Se implementaron cuatro fases secuenciales para cerrar esas brechas:

| Fase                     | Qué resuelve                                               | Estado                      |
|--------------------------|------------------------------------------------------------|-----------------------------|
| 1 — Catálogo de software | Identidad de software vs. GitHub Advisory Database         | ✅ Implementada y verificada |
| 2 — Jerarquía de activos | Separación Activo (host) / Componente (software instalado) | ✅ Implementada y verificada |
| 3 — Autenticación        | Login, JWT, contraseñas con hash                           | ✅ Implementada y verificada |
| 4 — RBAC + Scope         | 4 roles reales + alcance por asignación de activos         | ✅ Implementada y verificada |

El sistema quedó con **4 roles operativos** ( `ADMIN` , `SECURITY_ANALYST` , `ASSET_OWNER` , `AUDITOR` ), autenticación **JWT stateless**, un modelo de datos con `Asset` (activo real) y `SoftwareComponent` (software instalado) como entidades separadas, y un mecanismo de **scope** que restringe a `ASSET_OWNER` a ver y operar únicamente sobre los activos que tiene asignados — verificado extremo a extremo, incluyendo casos borde, mediante pruebas automatizadas (36 tests unitarios en verde) y pruebas en vivo con los 4 roles reales contra la aplicación corriendo en Docker.

Durante la implementación y, sobre todo, durante las rondas de revisión posteriores a cada fase, se detectaron y corrigieron **11 defectos reales** (detallados en la §8), incluyendo uno de severidad alta que afectaba a todo endpoint protegido del sistema. Ninguno llegó a quedar sin corregir.

## 2. Arquitectura y stack tecnológico

| Capa            | Tecnología                                                                                                                                               |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend         | Spring Boot 3.3.4, Java 21, Spring Security 6.3.3                                                                                                        |
| Autenticación   | JWT ( <code>io.jsonwebtoken:jjwt</code> 0.12.6), BCrypt ( <code>BCryptPasswordEncoder</code> )                                                           |
| Persistencia    | PostgreSQL 15, JPA/Hibernate ( <code>ddl-auto=validate</code> , sin migración automática), MapStruct 1.6.2                                               |
| Frontend        | React 18, TypeScript, Vite, Tailwind CSS, React Router, Axios, <code>@hello-pangea/dnd</code> (Kanban)                                                   |
| Automatización  | n8n (workflows de escaneo, correlación contra GitHub Advisory Database y CISA KEV)                                                                       |
| Infraestructura | Docker Compose ( <code>vuln_postgres</code> , <code>vuln_n8n</code> , <code>gef_backend</code> , <code>gef_frontend</code> , <code>vuln_pgadmin</code> ) |

Decisión de diseño transversal: **las migraciones de base de datos son manuales**, nunca automáticas. Hibernate solo *valida* que el esquema coincida con las entidades ( `spring.jpa.hibernate.ddl-auto=validate` ); los scripts en `init/*.sql` solo se ejecutan solos en un volumen de Postgres nuevo. Contra una base de desarrollo ya existente, cada fase requirió aplicar su script a mano antes de desplegar el backend con las entidades nuevas.

## 3. Fase 1 — Catálogo de software

### 3.1 Problema

El campo que identificaba el software instalado ( `software` + `ecosystem` ) era texto libre, sin normalizar ni validar contra ninguna fuente externa. Dos formas distintas de escribir el mismo paquete ( `"Express"` vs `"express "` ) no correlacionaban entre sí, y no había forma de saber si un componente cargado en el sistema correspondía a un paquete real con vulnerabilidades conocidas en GitHub Advisory Database.

### 3.2 Solución implementada

- **Tabla nueva `software_catalog`** ( `init/03-software-catalog.sql` ): catálogo de paquetes con `package_name` , `ecosystem` , `display_name` , `source` (por defecto `GITHUB_ADVISORY` ), poblado por el

workflow de n8n a partir de la ingesta real de GitHub Advisory. Incluye extensión `pg_trgm` e índice GIN para búsqueda por similitud (autocomplete).

- **Columnas nuevas en el componente de software:** `catalogued BOOLEAN` (si el paquete existe en el catálogo) y `url VARCHAR(400)` (Package URL, formato `pkg:<ecosystem>/<software>@<version>`).
- **Normalización en el backend** (`SoftwareComponentService`): `software` y `ecosystem` se normalizan a minúsculas y sin espacios en los extremos antes de guardar o comparar, de forma que variantes de mayúsculas/espacios colapsen al mismo valor.
- **Endpoint de autocomplete:** `GET /api/software-catalog?search=<texto>`, consumido por un componente `SoftwareAutocomplete` en el frontend que reemplaza el input de texto libre original.
- **Resolución automática de `catalogued` / `url`** en cada alta o edición de componente, comparando contra `software_catalog` por `(package_name, ecosystem)` exacto.

### 3.3 Verificación

Se confirmó en vivo, contra el catálogo real ya poblado por n8n (73+ paquetes reales de GitHub Advisory al momento de la verificación), que un componente creado con un paquete presente en el catálogo (`io.netty:netty-transport-sctp`, ecosistema `maven`) se marca correctamente `catalogued: true` y genera el `url` esperado (`pkg:maven/io.netty:netty-transport-sctp@4.1.100`). Los 6 componentes del set de datos semilla (`react`, `express`, `axios`, `lodash`, `spring-boot-starter-web`, `postgres`) no figuran en el catálogo importado y por lo tanto quedan correctamente `catalogued: false` — comportamiento esperado, no un defecto: el catálogo solo contiene paquetes con advisories reales, no todos los paquetes existentes.

## 4. Fase 2 — Jerarquía Activo / Componente

### 4.1 Problema

Lo que el sistema llamaba `assets` era en realidad un componente de software instalado (identificado por `software + ecosystem + version`), no un host o aplicación real. No existía ningún nivel que representara "el servidor X" o "la aplicación Y" como entidad propia — imprescindible para que, más adelante, el rol `ASSET_OWNER` tuviera algo concreto sobre lo cual ser responsable.

### 4.2 Solución implementada

`init/04-asset-hierarchy.sql` ejecuta, en un único script idempotente:

1. **Rename** de la tabla `assets` (componente de software) a `software_components`, incluyendo el rename manual de la *sequence* del `BIGSERIAL` (`assets_id_seq` → `software_components_id_seq`, paso que Postgres no hace automáticamente) y de sus constraints (`unique_asset_per_environment` → `unique_component_per_environment`, `fk_assets_environment` → `fk_components_environment`).
2. **Tabla `assets` nueva** = el activo real: `id, name, asset_type` (`SERVIDOR|APLICACION|VM|CONTENEDOR|ENDPOINT`), `environment_id, description, created_at`, con unicidad de `(name, environment_id)` — permite, por ejemplo, un host `api-gateway` en Testing y otro homónimo en Producción.
3. **Vínculo `software_components.asset_id`** (nullable, `ON DELETE SET NULL`): un componente sin host asignado sigue funcionando exactamente igual que antes del cambio; asignarlo a un host es opcional, no retroactivo

ni obligatorio.

En el código: la clase `Asset.java` original se renombró a `SoftwareComponent.java` (junto con su controller, repository, mapper y DTO), y se creó una clase `Asset.java` **nueva** para el host real, con su propio `AssetController` (`/api/assets`) siguiendo el mismo patrón que `EnvironmentController`.

En el frontend: `Inventory.tsx` pasó a representar los componentes de software (con selector opcional de host), y se agregó una pantalla nueva, `Assets.tsx`, para dar de alta y administrar los activos reales.

### 4.3 Defecto encontrado y corregido durante la revisión

`AssetService` (el del host nuevo) no tenía verificación de duplicados antes de insertar, a diferencia de `SoftwareComponentService`, que sí la tenía desde antes. Crear dos hosts con el mismo `(name, environment_id)` producía un **500 crudo**, con detalle de la constraint SQL filtrado en la respuesta al cliente. Se corrigió agregando un método `checkDuplicate()` (mismo patrón ya usado en `SoftwareComponentService` / `UserService`), que ahora devuelve un **409 Conflict** legible, con 4 tests de regresión.

### 4.4 Verificación

Confirmado en vivo: eliminar un host **no borra** sus componentes asociados — quedan sin host (`asset_id = NULL`), preservando los datos, en lugar de un `ON DELETE CASCADE`. Los duplicados con `environment_id` no nulo dan 409 limpio; con `environment_id` nulo no hay conflicto (la constraint UNIQUE de Postgres no colisiona sobre valores NULL), comportamiento documentado explícitamente en el código.

---

## 5. Fase 3 — Autenticación

---

### 5.1 Problema

GEF Secure no tenía **ningún** sistema de autenticación: no había dependencia de Spring Security, no había `SecurityFilterChain`, `UserController` era un CRUD abierto sin verificación de identidad, y el campo `role` de usuario existía en la base pero no lo leía ningún punto del código. Sin esto, aplicar RBAC en la Fase 4 no tenía nada sobre qué apoyarse.

### 5.2 Solución implementada

- **Spring Security + JWT stateless** (sin sesiones de servidor, sin OAuth/SSO externo — decisión deliberada para no exceder el alcance de una tesis de grado).
- `init/05-auth.sql`: agrega `users.password_hash`, y para los usuarios del seed (`fede.frankengerger`, `auditor.demo`) genera el hash BCrypt de una contraseña de desarrollo fija (`GefSecure2026!`, documentada como "solo entorno local") **directamente en SQL usando la extensión `pgcrypto`**, en el mismo formato que luego valida `BCryptPasswordEncoder` del lado Java, sin conversión intermedia.
- `POST /api/auth/login`, único endpoint público (`permitAll()` explícito en `SecurityConfig` — de lo contrario el propio login queda bloqueado por el filtro que se supone que abre).
- Token JWT con expiración de **480 minutos (8 horas)**, configurable por variable de entorno (`JWT_EXPIRATION_MINUTES`), firmado con clave simétrica (`JWT_SECRET`).

- Frontend: pantalla `Login.tsx`, contexto `AuthContext` con persistencia del token en `localStorage`, guarda de ruta `RequireAuth`, e interceptores de Axios en `services/api.ts` que agregan el header `Authorization: Bearer <token>` en cada request y, ante un 401 que no sea el propio login, limpian la sesión y redirigen a `/login`.

### 5.3 Defectos encontrados y corregidos durante la implementación

- El `AuthenticationEntryPoint` personalizado instanciaba su propio `ObjectMapper` (`new ObjectMapper()`), que no heredaba la configuración `spring.jackson.serialization.write-dates-as-timestamps=false` del resto de la aplicación — el timestamp del cuerpo de un 401 salía en formato array en vez de string ISO, inconsistente con el resto de los errores de la API. Corregido inyectando el `ObjectMapper` administrado por Spring en lugar de instanciar uno nuevo.
- Spring Boot generaba automáticamente un usuario en memoria con contraseña aleatoria en cada reinicio (ruido en los logs, sin ningún efecto real ya que la autenticación es 100 % manual vía JWT). Corregido excluyendo `UserDetailsServiceAutoConfiguration` en la clase principal.

### 5.4 Limitación documentada (decisión consciente, no omisión)

**No hay revocación de token.** Un JWT válido sigue siendo válido hasta su expiración natural (8 horas) aunque el usuario cierre sesión o un administrador lo dé de baja en ese lapso — no existe una blacklist de tokens ni un mecanismo de invalidación server-side. Se documenta como limitación explícita del alcance de tesis, no como un descuido: implementar revocación real (blacklist en Redis, tokens de corta duración + refresh token, etc.) es trabajo adicional fuera del alcance mínimo necesario para sostener RBAC + Scope.

### 5.5 Verificación

Confirmado en vivo: login con credenciales inválidas → 401 con mensaje genérico (no revela si el usuario existe); login válido → JWT con claims `sub` (username), `id`, `role`; requests sin token → 401; requests con token manipulado/inválido → 401; el interceptor del frontend limpia `localStorage` y redirige a `/login` automáticamente ante cualquier 401 que no sea el propio intento de login.

---

## 6. Fase 4 — RBAC + Scope

Es la fase de mayor superficie de cambio y la más sensible (autenticación/autorización), por lo que se trató con el mayor nivel de revisión manual del proyecto: plan aprobado explícitamente antes de escribir código, TDD real durante la implementación, y dos rondas de revisión escéptica posteriores (no solo una re-confirmación de lo ya hecho).

### 6.1 Modelo: rol define *qué*, asignación define *sobre qué*

Cuatro roles, sin roles por entorno ni por activo (`RESPONSABLE_PRODUCCION`, etc., deliberadamente descartados para evitar una explosión combinatoria difícil de mantener):

| Rol              | Descripción                                                                                                                                                                    |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ADMIN            | Control total: usuarios, asignaciones, configuración, y todo lo que puede el resto de los roles.                                                                               |
| SECURITY_ANALYST | Gestiona activos, componentes y vulnerabilidades; puede leer usuarios (para asignar responsables) pero no administrarlos.                                                      |
| ASSET_OWNER      | Solo ve y opera sobre los activos que tiene asignados explícitamente (tabla <code>user_asset_assignments</code> ). Puede mover el estado de sus vulnerabilidades en el Kanban. |
| AUDITOR          | Solo lectura, sin restricción de scope — ve todo el sistema, no puede modificar nada.                                                                                          |

`init/06-rbac.sql`:

```
ALTER TABLE public.users
  ADD CONSTRAINT check_users_role
  CHECK (role IN ('ADMIN','SECURITY_ANALYST','ASSET_OWNER','AUDITOR'));

CREATE TABLE IF NOT EXISTS public.user_asset_assignments (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT NOT NULL REFERENCES public.users(id) ON DELETE CASCADE,
  asset_id    BIGINT NOT NULL REFERENCES public.assets(id) ON DELETE CASCADE,
  assigned_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT unique_user_asset UNIQUE (user_id, asset_id)
);
```

## 6.2 Identidad completa en el JWT

El token, que en la Fase 3 solo llevaba `username` + `role`, se amplió con el `id` del usuario (`JwtService.generateToken(Long id, String username, String role)`), necesario para resolver el scope de un `ASSET_OWNER` sin una consulta extra a la base en cada request. Se introdujeron:

- `AuthenticatedPrincipal` (record `id, username, role`), que reemplaza al `String` que antes era el *principal* de `Authentication`.
- `CurrentUser`, helper estático que expone `get()` y `isAssetOwner()` leyendo el `SecurityContextHolder`, evitando inyectarlo a mano en cada service.
- `@EnableMethodSecurity` en `SecurityConfig`, habilitando `@PreAuthorize` a nivel de método en los controllers.

## 6.3 Scope: filtrado en la capa de servicio, con 404 en vez de 403

`AssetService`, `SoftwareComponentService` y `VulnerabilityAuditService` implementan el mismo patrón:

- **Listados** (`findAll` / `search`): si el usuario autenticado es `ASSET_OWNER`, se filtra el resultado contra `UserAssetAssignmentService.assignedAssetIds(userId)`; cualquier otro rol ve todo.
- **Acceso puntual** (`findById`, y en `VulnerabilityAuditService` también `updateStatus`): si el recurso pedido está fuera del scope de un `ASSET_OWNER`, se lanza `ResourceNotFoundException` → **404**, no 403 — decisión de diseño deliberada para no confirmarle a un usuario sin permiso que un recurso específico existe.
- El scope de `SoftwareComponent` se resuelve por `component.getAsset().getId()` (un componente sin host asignado queda invisible para `ASSET_OWNER`, coherente con la Fase 2); el de `VulnerabilityAudit` se resuelve transitivamente por el host del componente afectado.

`PATCH /api/vulnerabilities/{id}/status` se gatea a **nivel de endpoint completo** (rol), no campo por campo: un `ASSET_OWNER` dentro de su scope puede enviar `status`, `assignedTo` y `decision` en el mismo request, aunque conceptualmente sean acciones distintas. Simplificación consciente: ningún otro endpoint del sistema autoriza a nivel de campo, y separarlo habría cambiado el contrato que ya consume el Kanban sin aportar valor real de seguridad.

## 6.4 Matriz final de permisos por endpoint

| Controller                                | Método(s)                                                           | Roles permitidos                                                                                |
|-------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>AuthController</code>               | <code>POST /login</code>                                            | Público                                                                                         |
| <code>EnvironmentController</code>        | <code>GET</code>                                                    | Cualquier autenticado                                                                           |
| <code>EnvironmentController</code>        | <code>POST / PUT / DELETE</code>                                    | <code>ADMIN</code>                                                                              |
| <code>EnvironmentController</code>        | <code>POST /{id}/scan</code>                                        | <code>ADMIN</code> , <code>SECURITY_ANALYST</code>                                              |
| <code>AssetController</code>              | <code>GET</code>                                                    | Cualquier autenticado (con scope si <code>ASSET_OWNER</code> )                                  |
| <code>AssetController</code>              | <code>POST / PUT / DELETE</code>                                    | <code>ADMIN</code> , <code>SECURITY_ANALYST</code>                                              |
| <code>AssetController</code>              | asignaciones ( <code>GET / POST / DELETE /{id}/assignments</code> ) | <code>ADMIN</code>                                                                              |
| <code>SoftwareComponentController</code>  | <code>GET</code>                                                    | Cualquier autenticado (con scope si <code>ASSET_OWNER</code> )                                  |
| <code>SoftwareComponentController</code>  | <code>POST / PUT / DELETE / POST /{id}/scan</code>                  | <code>ADMIN</code> , <code>SECURITY_ANALYST</code>                                              |
| <code>VulnerabilityAuditController</code> | <code>GET</code>                                                    | Cualquier autenticado (con scope si <code>ASSET_OWNER</code> )                                  |
| <code>VulnerabilityAuditController</code> | <code>POST</code>                                                   | <code>ADMIN</code> , <code>SECURITY_ANALYST</code>                                              |
| <code>VulnerabilityAuditController</code> | <code>PATCH /{id}/status</code>                                     | <code>ADMIN</code> , <code>SECURITY_ANALYST</code> , <code>ASSET_OWNER</code> (dentro de scope) |
| <code>VulnerabilityAuditController</code> | <code>DELETE</code>                                                 | <code>ADMIN</code> , <code>SECURITY_ANALYST</code>                                              |
| <code>UserController</code>               | <code>GET</code>                                                    | <code>ADMIN</code> , <code>SECURITY_ANALYST</code>                                              |
| <code>UserController</code>               | <code>POST / PUT / DELETE</code>                                    | <code>ADMIN</code>                                                                              |
| <code>SystemErrorController</code>        | todo el controller                                                  | <code>ADMIN</code>                                                                              |
| <code>ScanController</code>               | <code>POST /scan</code>                                             | <code>ADMIN</code> , <code>SECURITY_ANALYST</code>                                              |
| <code>ScanController</code>               | <code>GET /scan-reports/latest</code>                               | Cualquier autenticado                                                                           |
| <code>SoftwareCatalogController</code>    | <code>GET</code> (autocomplete)                                     | Cualquier autenticado                                                                           |

**Reconciliaciones necesarias respecto de la matriz de permisos original**, encontradas al confrontarla contra el código real (la matriz original era anterior al split Host/Componente de la Fase 2 y a varios endpoints que hoy existen):

- `GET /api/users` no podía ser 100 % `ADMIN`: el Kanban lo usa para poblar el selector "Asignar responsable", y `SECURITY_ANALYST` también necesita poder asignar. Se separó por método: lectura para `ADMIN` + `SECURITY_ANALYST`, escritura solo `ADMIN`.
- `SystemErrorController` y las asignaciones de activo no estaban en la matriz original (son infraestructura de gestión del scope) → se asignaron a `ADMIN`, mismo nivel de sensibilidad que "Configurar sistema".
- No se migró `vulnerabilities_audit.assigned_to` a una foreign key real en esta fase — el endpoint queda protegido de la misma forma sea el campo texto libre o una referencia. Queda como trabajo futuro explícito.

## 6.5 Frontend: gating por rol

- `Settings.tsx`: gate completo ("Acceso restringido") si `role !== 'ADMIN'`; pestaña nueva "Asignaciones" (selector de activo + alta/baja de usuarios `ASSET_OWNER` asignados); selector de rol ampliado a los 4 roles reales (antes solo ofrecía `AUDITOR` / `ADMIN`).
- `Inventory.tsx` / `Assets.tsx` / `Scans.tsx`: botones de crear/editar/eliminar/escanear ocultos para roles sin ese permiso — no reemplaza el 403 real del backend, es solo UX.
- `Kanban.tsx`: `AUDITOR` no puede iniciar *drag* ni guardar cambios (solo lectura); se dejó de pedir `GET /api/users` quien no puede llamarlo.
- `Sidebar.tsx`: el link "Config" queda oculto para quien no sea `ADMIN`.
- Tipos y cliente HTTP nuevos: `UserAssetAssignment`, `assetAssignmentApi`.

## 6.6 Verificación

Cobertura de tests (Mockito, sin contexto de Spring, `SecurityContextHolder` seteado/limpiado a mano por test): scope filtrado para `ASSET_OWNER` vs. visión completa para el resto de los roles, `findById` fuera de scope → 404, en `AssetServiceTest`, `SoftwareComponentServiceTest` y un `VulnerabilityAuditServiceTest` nuevo.

Pruebas en vivo con los 4 roles reales contra la aplicación en Docker, incluyendo 9 casos borde adicionales verificados en la revisión final (componente sin host invisible incluso pidiéndolo por ID directo, doble asignación → 409, desasignar un usuario no asignado → 404, distinción entre CRUD de entorno —solo `ADMIN` — y escaneo de entorno — `ADMIN` + `SECURITY_ANALYST` —) y verificación visual con Playwright confirmando que cada rol ve exactamente los botones y datos que le corresponden.

## 7. Metodología de verificación aplicada

El criterio seguido en las cuatro fases, de forma consistente:

1. **Planificación explícita antes de programar**, con preguntas de diseño resueltas antes de escribir código (persistencia del token, contraseña de desarrollo, alcance del scope, granularidad del ganeo de `PATCH .../status`, entre otras).
2. **TDD real** para lógica de negocio no trivial: test que falla primero (rojo), implementación mínima para pasarlo (verde), un segundo caso con datos distintos para descartar una implementación *hardcodeada* (triangulación), y refactor manteniendo los tests en verde. No se aplicó a CRUD mecánico sin lógica propia.
3. **Build completo en verde** (`./gradlew build`) antes de dar una fase por terminada.

4. **Revisión escéptica posterior a cada fase**, explícitamente pedida como una revisión genuina y no una reconfirmación de lo ya hecho — relejendo el código con la pregunta "¿esto es correcto?", no "¿esto es lo que dije que iba a hacer?".
5. **Pruebas en vivo** contra la aplicación real corriendo en Docker (usuarios de prueba creados, operados y luego eliminados), cubriendo tanto el camino feliz como casos borde no ejercitados por los tests unitarios (interacción entre capas: `@PreAuthorize` + manejo de excepciones + scope de servicio, algo que un test de servicio con Mockito no puede detectar por sí solo).
6. **Verificación visual del frontend con Playwright**, logueando con cada rol real y confirmando en el DOM renderizado (no solo por código) qué se muestra y qué no.
7. **Limpieza de datos de prueba** al final de cada ronda de verificación, confirmando que la base vuelve al estado exacto del set semilla antes de continuar.

Este proceso —y en particular el paso 4— es el que permitió encontrar los 11 defectos listados en la sección siguiente: ninguno fue reportado por un usuario ni encontrado en producción, todos se detectaron en revisión interna antes de considerar el trabajo terminado.

---

## 8. Defectos encontrados y corregidos

| #  | Fase donde se detectó | Descripción                                                                                                                                                                                               | Corrección                                                                                                                                                                                      |
|----|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | 1                     | Postgres nativo de Windows en el puerto 5432 colisionaba con el contenedor <code>vuln_postgres</code>                                                                                                     | Remapeo de puerto a <code>5433:5432</code> en <code>docker-compose.yml</code>                                                                                                                   |
| 2  | 1                     | <code>Timezone</code> de la JVM ( <code>America/Buenos_Aires</code> , alias legado) no coincidía con el de Postgres ( <code>America/Argentina/Buenos_Aires</code> , IANA canónico)                        | Forzado <code>UTC</code> explícito en las tareas Gradle de test/ejecución                                                                                                                       |
| 3  | 2                     | <code>AssetService</code> (host) sin verificación de duplicados → 500 crudo con detalle SQL filtrado                                                                                                      | Agregado <code>checkDuplicate()</code> → 409 legible, con tests de regresión                                                                                                                    |
| 4  | 3                     | <code>ObjectMapper</code> propio en el <code>AuthenticationEntryPoint</code> no heredaba la config de timestamps → formato inconsistente en el 401                                                        | Injectado el <code>ObjectMapper</code> administrado por Spring                                                                                                                                  |
| 5  | 3                     | Spring Boot autogeneraba un usuario en memoria con contraseña aleatoria en cada reinicio (ruido sin efecto real)                                                                                          | Excluida <code>UserDetailsServiceAutoConfiguration</code>                                                                                                                                       |
| 6  | 4 (implementación)    | <b><code>AccessDeniedException</code> sin handler dedicado → 500 con mensaje interno filtrado en cualquier endpoint protegido por <code>@PreAuthorize</code> de todo el sistema</b>                       | Agregado <code>@ExceptionHandler(AccessDeniedException.class)</code> → 403 limpio. Defecto más grave detectado en todo el proceso: afectaba sistémicamente a toda la superficie de autorización |
| 7  | 4 (implementación)    | <code>DELETE</code> de vulnerabilidades configurado por error como <code>ADMIN</code> - only, en contradicción con el plan aprobado ( <code>ADMIN</code> + <code>SECURITY_ANALYST</code> )                | Autodetectado durante la implementación y corregido antes de cerrar la fase                                                                                                                     |
| 8  | 4 (revisión final)    | Ruta <code>/assets</code> (página "Activos") colisionaba con el directorio <code>dist/assets/</code> de Vite; nginx la interceptaba devolviendo 301→403 en toda navegación directa o <code>refresh</code> | Bundles movidos a <code>dist/static/</code> ( <code>vite.config.ts</code> + <code>nginx.conf</code> )                                                                                           |
| 9  | 4 (revisión final)    | <code>Settings.tsx</code> disparaba sus <code>fetch</code> de datos aunque el gate de <code>ADMIN</code> mostrara "Acceso restringido", generando toasts de error 403 para el resto de los roles          | Efectos de carga condicionados a <code>isAdmin</code>                                                                                                                                           |
| 10 | 4 (revisión final)    | <code>Kanban.tsx</code> llamaba <code>GET /api/users</code> (solo <code>ADMIN</code> / <code>SECURITY_ANALYST</code> ) también para <code>ASSET_OWNER</code> ; al usar                                    | Excluida la llamada también para <code>ASSET_OWNER</code>                                                                                                                                       |

| #  | Fase donde se detectó | Descripción                                                                                                                                                                            | Corrección                                           |
|----|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
|    |                       | <code>Promise.all</code> , el 403 rompía la carga completa del tablero                                                                                                                 |                                                      |
| 11 | 4 (revisión final)    | Alta de usuario nueva completamente rota: el formulario nunca tuvo campo de contraseña pese a que el backend la exige al crear                                                         | Agregado el campo (obligatorio solo al crear)        |
| 12 | 4 (revisión final)    | Botón "Eliminar" visible para <code>ASSET_OWNER</code> en el modal del Kanban pese a que el backend restringe <code>DELETE</code> a <code>ADMIN</code> / <code>SECURITY_ANALYST</code> | Botón ocultado también para <code>ASSET_OWNER</code> |

## 9. Estado final verificado

- **Backend:** `./gradlew build` en verde desde una compilación limpia ( `clean test` , sin caché) — **36 tests, 0 fallos** al cierre de la Fase 4, distribuidos en 7 clases de test ( `JwtServiceTest` , `AssetServiceTest` , `AuthServiceTest` , `SoftwareCatalogServiceTest` , `SoftwareComponentServiceTest` , `VulnerabilityAuditServiceTest` , `BackendApplicationTests` ). Ese número subió a 72 tras las mejoras de §11 (§11.10/§11.12), bajó a 69 en §11.14 (al reemplazar 5 tests de un mecanismo eliminado por 2 más simples y directos, sin pérdida de cobertura), y subió a **71** en §11.17 (2 tests nuevos para el escaneo por Activo).
- **Frontend:** build de producción ( `vite build` ) sin errores de TypeScript, contenedor Docker reconstruido con la última versión del código.
- **Integración n8n:** workflow de escaneo disparado end-to-end en sus cuatro tipos (GLOBAL, ENTORNO, ACTIVO y ACTIVO con ecosistema no reconocido por GitHub) y en el disparador programado diario, reporte generado correctamente, sin errores registrados en el log del sistema (ver detalle en §11.13).
- **Base de datos:** las cuatro migraciones ( `init/03` a `init/06` ) aplicadas contra el entorno de desarrollo; estado verificado limpio (coincide exactamente con el set de datos semilla) tras cada ronda de pruebas.
- Los cuatro roles fueron probados en vivo, uno por uno, confirmando qué pueden y qué no pueden ver ni hacer, tanto a nivel de API (curl) como de interfaz renderizada (Playwright).
- **Catálogos de Config** (§11.11): `GET/POST/PUT/DELETE /api/asset-types` y `/api/ecosystems` verificados en vivo (200, escritura `ADMIN` -only), y las 3 pestañas nuevas de Config probadas de punta a punta con Playwright (creación/edición/borrado, propagación a los `<select>` de Activos y Componentes, limpieza de datos de prueba).
- **Auditoría de datos congelados vs. en vivo** (§11.12): los 3 casos reproducidos y corregidos en vivo — divergencia real de entorno en un componente existente, estado de un hallazgo desincronizado del Kanban (reproducido cambiando un CVE real a "RESUELTA" y confirmando el reflejo en el Historial), y resolución por FK sobreviviendo a un rename simulado que habría roto la búsqueda por nombre.
- **Correlación real de escaneos con GitHub Advisory** (§11.13): GLOBAL (31 hallazgos), ENTORNO (27), ACTIVO normal (3) y ACTIVO con ecosistema no reconocido (0 hallazgos falsos, sin quedar trabado) — los cuatro con datos reales, verificados contra la base. Disparador programado diario confirmado con logging interno de n8n mostrando el cron registrado y ejecutado puntualmente dos veces consecutivas.

- **Detalle de escaneo por `scan_id` real** (§11.14): reportado en vivo por el usuario ( `SCN-2026-000033` ) mostraba 14 vulnerabilidades en vez de 3) y corregido; verificado que ahora coincide exactamente con lo detectado por cada escaneo, que el comparador (§11.9) sigue funcionando con datos reales, y que el caso límite de §11.13 (componente sin ecosistema reconocido) muestra 0 en vez de heredar resultados de otro escaneo.
- **Drill-down por activo en GLOBAL/ENTORNO** (§11.15): verificado con Playwright en los dos lugares que comparten el patrón — Historial ( `SCN-2026-000034` , click en "Lodash Utils" mostró sus 4 CVE exactas) y el resultado en vivo de la pestaña "Escaneos" (escaneo ENTORNO real sobre "Desarrollo") — sin errores de consola en ninguno de los dos.
- **Flujo inyector** (§11.16): botón "Inyectar Activo" (ADMIN-only, confirmado con Playwright contra los 3 roles) crea un host de prueba con 5 componentes de software real de severidad variada ( `low` / `medium` / `high` / `critical` , nunca todo `CRITICAL` ) y entorno al azar; un escaneo GLOBAL real sobre el host inyectado detectó y auditó 27 hallazgos reales con prioridades calculadas por el motor de riesgo. Se detectó y corrigió en el camino que los contenedores de frontend y backend estaban sirviendo imágenes desactualizadas — reconstruidos ambos y reverificado de punta a punta.
- **Escaneo por Activo (host) completo** (§11.17): nuevo modo "Activo" en Centro de Escaneos, reportado como gap por el usuario al no poder escanear el host inyectado sin ir componente por componente. Verificado en vivo: `ScanReport` con FK real a `assets` , hallazgos de 4 componentes distintos en un solo resultado con columna "Componente", y el caso de homónimos (2 Activos con el mismo nombre en entornos distintos) confirmado sin contaminación cruzada.
- **Pulido de UX en Centro de Escaneos** (§11.18): filtros de Historial y "Nuevo escaneo" ajustados al contenido (sin espacio vacío) y en una sola fila; botón "Limpiar" agregado a Escaneos (con una corrección de paso en `useScanPolling` para que no dejara "Escanear" deshabilitado); tabla de resultados visible desde el inicio con sus encabezados; popup informativo cuando un escaneo termina sin hallazgos, distinto del mensaje de "todavía no se corrió ningún escaneo" — verificado en vivo con Playwright en cada paso.
- **Dashboard: KPIs compactos** (§11.19): las 7 tarjetas de KPI pasaron a una sola fila (ícono arriba, texto centrado debajo, `p-3` en vez de `p-5` ), liberando espacio vertical para los gráficos. Verificado con Playwright en 1280px y 1536px, sin texto cortado ni desbordado en ninguna de las dos resoluciones.

## 10. Limitaciones conocidas y trabajo futuro

Documentadas explícitamente como decisiones de alcance, no como omisiones:

- **Sin revocación de token** (Fase 3, §5.4): un JWT sigue siendo válido hasta su expiración natural aunque se cierre sesión o se dé de baja al usuario.
- **Autenticación mínima viable**: sin recuperación de contraseña por email, sin 2FA, sin políticas de expiración configurables por rol.
- `asset_vulnerabilities.assigned_to` **sigue siendo texto libre** (antes en `vulnerabilities_audit` , movido en §11.8), no una foreign key a `users.id` — no era necesario para que el gateo por rol funcione, pero queda pendiente para una futura fase si se quiere integridad referencial completa sobre el responsable asignado.
- **Multi-asignación sin jerarquía de prioridad**: un mismo activo puede tener varios `ASSET_OWNER` asignados, sin ningún mecanismo de "responsable principal".

- ~Pipeline de n8n sin correlación real de advisories (§11.6)~ — **resuelto en §11.13:**  
`Fetch_GitHub_Advisories` ahora consulta por paquete/ecosistema realmente instalado en vez de traer un lote genérico sin filtrar.
- **Log de auditoría separado** (Etapa 6 de la propuesta de trazabilidad, §11.9): la propia propuesta la condicionaba a que surgiera una pregunta de auditoría concreta que `Scan` + `AssetVulnerability` no pudieran responder — todavía no apareció esa necesidad en uso real, así que se dejó sin implementar en vez de construir algo especulativo.
- **AssetType / Ecosystem sin FK real hacia assets / software\_components** (§11.11): son catálogos de sugerencia para los `<select>` de alta, no relaciones referenciales — decisión consciente para no migrar columnas de texto libre existentes sin necesidad concreta; el sistema tampoco valida esos valores hoy.
- **n8n/bootstrap.sh frágil ante logging en modo debug** (§11.13): `publish_all()` extrae IDs de workflow filtrando la salida de `n8n list:workflow` por el separador `|`; con `N8N_LOG_LEVEL=debug` el CLI también emite líneas de log con ese mismo separador, y el script puede tomar una de ellas como si fuera un ID, dejando el workflow desactivado sin ningún error visible. No afecta al nivel de log de producción (confirmado), por eso no se corrigió en esta pasada — queda documentado para no repetir el diagnóstico si vuelve a hacer falta depurar con logging detallado.
- Todo lo ya declarado fuera de alcance en los informes originales (SSO/OAuth externo, CPE/SBOM, soporte multi-tenant) se mantiene fuera de alcance.

---

## 11. Mejoras posteriores a la Fase 4

---

Cambios implementados **después** de cerrar la Fase 4, ya con autenticación y RBAC funcionando sobre el sistema. No corresponden a una fase nueva del plan original — son refinamientos de UX y de modelo de datos que surgieron al usar la aplicación con los cuatro roles reales ya operativos. Se documentan separados del cuerpo principal (§1–§10) porque no estaban contemplados en el plan unificado, pero se incluyen en el mismo informe porque terminan de definir el estado actual del sistema.

Mismo criterio de trabajo que en las fases anteriores: para los cambios de UX o de datos sin impacto en integridad (la mayoría de esta sección) se armó una propuesta y se ajustó con el usuario antes de tocar código; para los que sí tocaban borrado o migración de datos existentes se entró en modo de planificación explícito y se resolvieron por adelantado las preguntas de diseño con mayor impacto antes de escribir una sola línea.

### 11.1 Rediseño del Centro de Escaneos

**Problema.** La pantalla de Escaneos tenía tres tarjetas de disparo independientes (por componente / por entorno / global) y una tabla "Vulnerabilidades detectadas" que en realidad listaba *todas* las vulnerabilidades del sistema, sin relación con el escaneo recién ejecutado. El resultado real de un escaneo (`ScanReportDTO`: total detectado, desglose por severidad, estado del sistema) ya lo calculaba el backend y ya llegaba al frontend vía `GET /api/scan-reports/latest`, pero se descartaba — solo se mostraba un toast genérico de "completo".

**Solución implementada.**

- **Tarjeta de disparo unificada:** reemplaza las tres tarjetas por una sola con selector Componente/Entorno/Global, que muestra el campo correspondiente (dropdown de componente, de entorno, o nada si es global) y un único botón "Escanear".
- **Resultado del escaneo puntual:** al terminar, se muestra lo que ese escaneo encontró — para un componente, la lista de sus vulnerabilidades (o "No se encontraron vulnerabilidades para este activo."); para entorno/global, la misma información pero **agrupada por activo afectado** (activo, cantidad de hallazgos, severidad más alta), calculada del lado del cliente.
- **Mecanismo de correlación:** `VulnerabilityAuditService.create()` siempre inserta una fila nueva, nunca actualiza una existente (no hay upsert por CVE) — eso permite identificar exactamente lo que un escaneo encontró filtrando por `detectedAt >= momento en que se disparó el escaneo`, sin tocar el esquema. Se combina con el alcance del objetivo (`assetId` o `environmentId`) para evitar cruces si dos escaneos corrieran en paralelo. Nuevo endpoint `GET /api/vulnerabilities/scan-result?since=&assetId=&environmentId=`.
- **Pestaña "Historial":** tabla con todas las corridas de escaneo pasadas (`scan_reports`), con filtros por fecha, tipo de objetivo y objetivo específico. Nuevo endpoint `GET /api/scan-reports?targetType=&targetName=&from=&to=`, capado a los 200 resultados más recientes (mismo criterio que ya usaba `SystemErrorController` con su tope de 50 — no hay paginación real en ningún endpoint del proyecto, no se introdujo acá tampoco).
- Se sacó la tabla "Vulnerabilidades detectadas" de esta pantalla — la gestión de vulnerabilidades ya vive en el Kanban/Auditoría; duplicarla acá no aportaba.

**Defecto encontrado y corregido.** El filtro de historial con parámetros opcionales (`WHERE (:param IS NULL OR campo = :param)`) fallaba con `could not determine data type of parameter $N` — PgJDBC no puede inferir el tipo de un bind usado únicamente en una comparación `IS NULL`, es una limitación del protocolo extendido de Postgres, no un error de JPQL. Se corrigió con una query nativa (`nativeQuery = true`) con `CAST(:param AS tipo)` explícito en cada ocurrencia del parámetro.

**Verificación.** Confirmado en vivo: escaneo de un componente sin hallazgos nuevos → mensaje de vacío; escaneo de entorno con datos reales insertados durante la ventana de polling → tabla agrupada correcta (2 vulnerabilidades en un activo, 1 en otro, cada uno con su severidad más alta); filtros del historial por tipo y objetivo funcionando tras el fix; `AUDITOR` / `ASSET_OWNER` no ven la tarjeta de disparo pero sí el historial (lectura abierta).

## 11.2 Unificación de Inventario y Activos

**Problema.** `Inventario` y `Activos` eran dos pantallas separadas que en realidad gestionaban los dos niveles de la misma jerarquía (`Asset` = host real, `SoftwareComponent` = software instalado), sin conexión entre sí: no había forma de partir de un activo y ver qué software tenía instalado, ni de crear un host y cargarle software desde el mismo lugar. Además había una colisión de nombres real: `Inventory.tsx` (que gestionaba `SoftwareComponent`) usaba el texto "Activo" en su título, sus botones y sus toasts — resabio de antes de la Fase 2, cuando `Asset` todavía no existía como entidad separada.

### Solución implementada.

- Una sola pantalla "**Activos**": tabla de activos arriba (con columna nueva "Componentes" = cantidad de software instalado); al hacer click en una fila se abre debajo el detalle de ese activo con su software y un botón "Agregar Software" que abre el alta de componente con el host ya precargado.

- Corrección de nomenclatura: los toasts y títulos que decían "Activo" para una operación sobre un *componente* pasan a decir "Componente" ( "**Componente creado**" , "**Nuevo Componente**" , etc.), sin ambigüedad con las operaciones reales sobre el host.
- Se eliminó `pages/Inventory.tsx` ; la ruta `/inventory` redirige a `/assets` ; se sacó la entrada "Inventario" del sidebar.
- Sin cambios de backend: la lista completa de componentes ya se traía en `Inventory.tsx` , el filtrado por activo seleccionado se resuelve del lado del cliente (mismo criterio que el agrupamiento de §11.1).

**Verificación.** Confirmado en vivo: seleccionar un host con software muestra su detalle correctamente; crear un activo nuevo → seleccionarlo → agregarle software → aparece asociado; gate por rol sin cambios ( `ADMIN` / `SECURITY_ANALYST` con escritura, resto solo lectura); `/inventory` redirige y ya no aparece en el sidebar.

### 11.3 Software siempre con activo + borrado lógico en cascada

**Problema.** Al unificar Inventario y Activos quedó a la vista que el software podía quedar "suelto" ( `asset_id = NULL` ) — 5 de los 6 componentes semilla estaban así. Se decidió que el software **siempre** debe pertenecer a un activo. Consecuencia directa: borrar un activo con software ya no podía simplemente desvincularlo (comportamiento anterior) — había que decidir qué pasa con el software al borrar su host.

**Decisión de diseño (borrado lógico en cascada).** En vez de bloquear el borrado o borrar todo físicamente, se optó por borrado lógico: al eliminar un activo, tanto el activo como su software activo en ese momento quedan marcados con `deleted_at` (mismo timestamp para ambos), sin borrar nada físicamente — se preserva el historial de vulnerabilidades acumulado. Un `ADMIN` o `SECURITY_ANALYST` (los mismos roles que ya pueden borrar activos) puede restaurarlo después.

**Restauración selectiva por timestamp.** Restaurar un activo revive **únicamente** el software que se borró en ese mismo cascade (mismo `deleted_at` exacto que el activo) — un componente borrado independientemente antes o después de esa fecha no revive por accidente. Se implementa comparando el `deletedAt` del activo contra el `deletedAt` de cada componente, sin necesidad de una columna adicional de "grupo de borrado". Verificado en vivo con un caso real: componente A borrado junto con su host → ambos restaurados correctamente; componente B del mismo host borrado *independientemente* antes de borrar el host → no reaparece al restaurar.

**Migración de datos existentes.** Los 5 componentes huérfanos se promovieron, cada uno, a un activo nuevo con su mismo nombre y entorno ( `init/08-soft-delete-assets.sql` ), preservando el dato semilla en vez de inventar agrupaciones arbitrarias.

#### Cambios de esquema:

```
ALTER TABLE public.assets ADD COLUMN deleted_at TIMESTAMP WITHOUT TIME ZONE;
ALTER TABLE public.software_components ADD COLUMN deleted_at TIMESTAMP WITHOUT TIME ZONE;
ALTER TABLE public.software_components ALTER COLUMN asset_id SET NOT NULL;

-- Índice unico parcial: permite reusar el nombre de un activo ya eliminado logicamente
ALTER TABLE public.assets DROP CONSTRAINT unique_asset_name_per_environment;
CREATE UNIQUE INDEX unique_active_asset_name_per_environment
  ON public.assets (name, environment_id) WHERE deleted_at IS NULL;
```

**Endpoints nuevos:** `GET /api/assets/deleted` y `POST /api/assets/{id}/restore` , con los mismos roles que ya podían borrar ( `ADMIN` , `SECURITY_ANALYST` ) — no tendría sentido que solo `ADMIN` pudiera deshacer un borrado

que `SECURITY_ANALYST` también puede hacer.

**Frontend:** la pantalla de Activos se dividió en dos pestañas, "**Vigentes**" y "**Eliminados**" (esta última solo visible para `ADMIN` / `SECURITY_ANALYST`), para no confundir el estado "no eliminado" con el nombre genérico de la página ("Activos"). El selector de host en el alta de componente pasó a ser un campo obligatorio, sin la opción "Sin host".

**Defecto operativo encontrado (no de código).** Al aplicar la migración por primera vez contra la base de desarrollo usando redirección de archivo (`psql < archivo.sql`) en el entorno Windows/Git-Bash, el bloque de backfill insertó una sola fila en vez de las cinco esperadas, dejando los componentes huérfanos mal agrupados bajo un mismo activo. Reaplicar la misma lógica vía heredoc (`psql <<'EOF' ... EOF`) produjo el resultado correcto. Se documenta como advertencia operativa para futuras migraciones manuales en este entorno: preferir heredoc a redirección de archivo.

**Verificación.** `./gradlew build` completo en verde (ver §11.10). Flujo completo en vivo: crear activo → agregar software → eliminar (desaparece de "Vigentes") → aparece en "Eliminados" con la cantidad correcta de componentes afectados → restaurar → reaparece con su software intacto en "Vigentes". Rechazo de alta de componente sin `assetId` verificado (400, "**El activo es requerido**"). Gate por rol de la pestaña "Eliminados" confirmado (`SECURITY_ANALYST` la ve, `AUDITOR` no).

## 11.4 Vulnerabilidades del escaneo seleccionado (Historial)

**Problema.** El historial de escaneos (§11.1) mostraba métricas agregadas por corrida (total, críticas, altas, medias, bajas) pero no permitía ver *cuáles* vulnerabilidades puntuales había encontrado un escaneo pasado al hacer click en su fila.

### Solución implementada.

- Nuevo método `VulnerabilityAuditService.findByScanReport(reportId)`: resuelve el objetivo del reporte clickeado (`targetType` / `targetName` → `assetId` o `environmentId` real) y acota la ventana de correlación entre el reporte inmediatamente anterior del mismo objetivo (excluyente; sin techo si es el primero) y el reporte clickeado (inclusive) — mismo mecanismo de correlación por `detectedAt` que ya usaba el resultado de escaneo en vivo (§11.1), extendido con un límite superior (`until`) opcional.
- Nuevo endpoint `GET /api/scan-reports/{id}/vulnerabilities`.
- Frontend: fila clickeable en la tabla de Historial (mismo patrón `onRowClick` / `selectedKey` ya usado en Activos), que muestra el detalle de esa corrida puntual.

**Defecto encontrado y corregido.** Al resolver la ventana para el *primer* reporte de un objetivo (sin reporte anterior), se usaba `LocalDateTime.MIN` como centinela de "sin límite inferior" — Postgres no admite ese valor como `timestamp` (`timestamp out of range`, año ≈ -999.999.999). Se corrigió rediseñando `VulnerabilityAuditRepository` con tres queries nativas (`findByComponentScanWindow` / `findByEnvironmentScanWindow` / `findByGlobalScanWindow`) que aceptan `since` / `until` genuinamente nulos vía `CAST(:param AS timestamp) IS NULL OR ...` — mismo patrón ya usado para el filtro opcional del historial (§11.1), evitando cualquier valor centinela.

**Verificación.** Confirmado en vivo: click en una fila ACTIVO sin escaneo anterior para ese objetivo (el caso que disparaba el bug) → 200 con la vulnerabilidad correcta en vez de 500; casos ENTORNO y GLOBAL con mensaje de vacío correcto cuando no hay hallazgos; toggle de selección al hacer click de nuevo sobre la misma fila.

## 11.5 Paneles de detalle como popup (Historial y Activos)

**Problema.** Tanto el detalle de un escaneo en Historial (§11.4) como el detalle de software de un activo (§11.2) se mostraban como un panel fijo debajo de la tabla principal — con listas largas quedaba fuera de la vista sin hacer scroll, dando la sensación de que el contenido "se perdía".

**Solución implementada.** Se reemplazaron ambos paneles por el componente `Modal` ya existente en el proyecto (hasta entonces usado solo para formularios de alta/edición), que abre como popup centrado sobre un fondo oscurecido y se puede cerrar con el botón X, la tecla Escape o un click fuera. Se agregó un tamaño `x1` (`max-w-5x1`) al `Modal` para el detalle de Activos, cuya tabla de componentes tiene 8 columnas y no entraba sin scroll horizontal en el ancho `lg` existente.

**Verificación.** Confirmado en vivo con Playwright: apertura y cierre por las tres vías; al cerrar se deselectiona la fila subyacente en ambas pantallas; tabla de componentes de Activos visible completa sin scroll horizontal en el tamaño `x1`.

## 11.6 Unificación de prioridad de vulnerabilidades a inglés

**Problema.** La columna `vulnerabilities_audit.priority` tenía datos en dos idiomas: los datos sembrados usaban español (`CRITICA/ALTA/MEDIA/BAJA`), mientras que buena parte del código solo reconocía inglés (`CRITICAL/HIGH/MEDIUM/LOW`) — el nodo `Compute_Audit_Metrics` del pipeline de `n8n`, `DashboardService.countByPriority("CRITICAL")`, y `VulnerabilityCard.tsx` del Kanban. Efecto observable: el contador de "Vulnerabilidades Críticas" del Dashboard daba 0, las tarjetas del Kanban con prioridad en español no mostraban color de severidad, y los contadores del historial de escaneos no coincidían con el detalle de cada corrida (§11.4).

**Solución implementada.** Se unificó `priority` a inglés en todo el sistema: valores sembrados (`init/02-seed-data.sql`) y datos ya existentes en la base migrados a `CRITICAL/HIGH/MEDIUM/LOW`; se eliminó el mapeo dual español/inglés del frontend (`Scans.tsx`, tipo `VulnPriority`). Se recalcularon además los 15 reportes de escaneo ya guardados, reutilizando el propio endpoint de detalle (§11.4) para cada uno y persistiendo el total y desglose real por severidad — así el historial y el detalle de cada corrida coinciden exactamente.

**Limitación encontrada en esta pasada (resuelta después, §11.13).** En el momento de esta corrección, los escaneos *nuevos* seguían persistiendo esos contadores en 0, por una causa distinta y más profunda que el idioma: el nodo `Fetch_GitHub_Advisories` del pipeline de `n8n` consultaba `GET /advisories` de GitHub sin ningún filtro, trayendo un lote genérico de advisories recientes sin relación con el inventario real (`software_components`), por lo que el cruce contra el inventario casi nunca encontraba coincidencias. Confirmado revisando la ejecución del pipeline nodo por nodo en la UI de `n8n`: en ninguna de las ejecuciones registradas durante las pruebas de esta sesión se insertó un hallazgo real vía el pipeline (los únicos datos en `vulnerabilities_audit` seguían siendo los 9 del seed). Se dejó documentado como limitación en ese momento y se corrigió más adelante en la misma sesión de mejoras posteriores — ver §11.13 para la solución (filtrado por paquete/ecosistema instalado) y la verificación con hallazgos reales.

**Verificación.** Confirmado en vivo: Dashboard "Vulnerabilidades Críticas" pasó de 0 a 2; tarjetas del Kanban con color de severidad correcto para las 9 vulnerabilidades sembradas; los 15 reportes del historial muestran contadores reales, coincidentes con su propio detalle (incluida la fila con 2 críticas marcada " ⚠️ Acción requerida").

## 11.8 Trazabilidad completa de escaneos (Scan real + AssetVulnerability)

**Problema.** Hasta este punto, `scan_reports` no tenía ninguna relación real con `vulnerabilities_audit` — "qué encontró un escaneo puntual" se reconstruía con la heurística de ventana de tiempo de §11.4, y el ciclo de vida de una vulnerabilidad (`status`, `decision`, `resolved_at`) vivía en cada fila de detección individual, no en una entidad persistente por (componente, CVE). Consecuencia concreta: marcar un CVE como resuelto no "recordaba" nada si el mismo CVE volvía a detectarse en un escaneo posterior — se gestionaba como un caso nuevo, sin ningún vínculo con el anterior.

**Análisis previo.** Se armó un documento de análisis y propuesta aparte (`Propuesta_Trazabilidad_Escaneos_GEF_Secure.md`), con diagnóstico del modelo actual, diagrama de entidades (Mermaid), esquema de base de datos, plan de migración en 6 etapas y análisis de alternativas — antes de tocar una sola línea de código. Se implementaron las primeras 4 etapas en esta pasada (comparación entre escaneos y log de auditoría quedan para una sesión futura).

**Decisión de alcance.** En vez de renombrar clases y tablas (`ScanReport` → `Scan`, `VulnerabilityAudit` → `ScanFinding`, propuesta original), se optó por **extender las clases y rutas REST existentes** — misma arquitectura final, sin el costo de un rename en cascada sobre imports, tests y mappers que no aportaba funcionalidad. `/api/scan-reports/**` y `/api/vulnerabilities/**` no cambiaron de firma.

**Etapas 1 — Scan real.** `scan_reports` gana `public_code` (`SCN-2026-000018`), `status` (`RUNNING` / `COMPLETED`), `started_at`, `triggered_by`, y FKs resueltas a `software_components` / `environments` en el momento del disparo (no por nombre en cada lectura, como hacía `findByScanReport`). El backend crea el `Scan` en `RUNNING` antes de llamar a n8n y le pasa su `id` en el payload del webhook; n8n lo reenvía entre sus propios nodos y, al terminar, llama de vuelta a un endpoint nuevo (`POST /api/webhook/scan-report`) que completa el `Scan` — reemplazando el INSERT directo por SQL que hacía antes. Ese endpoint es interno (n8n no es un usuario del sistema): se protege con un token compartido por header (`X-Internal-Token`), no con JWT de usuario.

**Etapas 2 — scan\_id en cada hallazgo.** `vulnerabilities_audit` gana una FK real `scan_id`. El INSERT de n8n (`Persist_Audit_Findings`) pasa a incluirla, tomándola del mismo payload que ya trae el `Scan.id`. Reemplaza la heurística de ventana de tiempo para los datos nuevos — esa heurística se conserva para backfill de datos históricos y para `findByScanReport` (detalle de un escaneo pasado en el Historial, §11.4), que no se tocó.

**Etapas 3 — AssetVulnerability y ciclo de vida.** Tabla nueva: una fila por (componente, CVE), con el estado persistente a través del tiempo. Al completarse un `Scan`, `VulnerabilityAuditService.applyLifecycle()` recorre sus hallazgos y, por cada uno: crea el `AssetVulnerability` si es la primera vez que se ve ese CVE en ese componente (**NEW**), actualiza la fecha de última detección si ya estaba abierto (**PERSISTING**), o lo reabre e incrementa un contador si había sido resuelto (**REOPENED**). Al final, cualquier `AssetVulnerability` que seguía abierto en el mismo alcance del escaneo (mismo componente / mismo entorno / global, según el tipo) y no recibió ningún hallazgo nuevo pasa a **RESOLVED** automáticamente — es la pieza que antes no existía en absoluto: hoy el sistema puede detectar por sí solo que una vulnerabilidad dejó de aparecer, sin comparación manual entre escaneos.

**Etapas 4 — Kanban y Dashboard sobre AssetVulnerability.** `findAll`, `findByStatus`, `findById`, `updateStatus` y `delete` de `VulnerabilityAuditService` pasan a operar sobre `AssetVulnerability` en vez de sobre el hallazgo puntual — `vulnerabilities_audit` queda verdaderamente inmutable, nunca se vuelve a escribir después del INSERT inicial. Los conteos del Dashboard (críticas, MTTR, distribución por severidad/estado, tendencia de 30 días) migraron de igual forma. El contrato de la API no cambió: el mapper gana un segundo método (`AssetVulnerability` → el mismo DTO que ya consumían Kanban y Dashboard), verificado en vivo como visualmente idéntico antes y después del cambio.

**Decisión de diseño: dos ejes de estado separados.** Durante la implementación de la Etapa 4 se detectó que un solo campo `status` en `AssetVulnerability` mezclaba dos conceptos distintos: si los escaneos lo siguen encontrando (algo que decide el sistema) y en qué paso del triage humano está (algo que decide el analista en el Kanban). Se separó en `detectionStatus` (`OPEN` / `RESOLVED`, manejado exclusivamente por `applyLifecycle`) y `triageStatus` (`DETECTADA` / `EN_ANALISIS` / `RESUELTA`, manejado por el analista vía `updateStatus`, es lo único que expone la API). La resolución automática por ausencia **nunca** toca `triageStatus` — un analista investigando algo activamente no ve su caso cerrado solo porque el próximo escaneo no volvió a encontrarlo. Si el CVE reaparece (`REOPENED`), sí se fuerza `triageStatus` de vuelta a `DETECTADA`, para que no quede oculto como "resuelto" en el Kanban mientras en los hechos volvió.

### Defectos encontrados y corregidos.

- Al hacer el primer `INSERT` de `ScanReport` gestionado por Hibernate (antes siempre lo hacía n8n por SQL directo con `::jsonb` explícito), la columna `environment_breakdown` (`jsonb`) falló por mismatch de tipo — Hibernate bindea un `String` como `varchar` salvo que se declare explícitamente `@JdbcTypeCode(SqlTypes.JSON)`.

- n8n bloquea el acceso a `$env` en expresiones por defecto (`N8N_BLOCK_ENV_ACCESS_IN_NODE`), lo que impedía que el nodo nuevo leyera el token interno — se habilitó explícitamente en la configuración del contenedor.

**Verificación.** `./gradlew build` en verde (ver §11.10). Confirmado en vivo, extremo a extremo: un escaneo real dispara `Scan` en `RUNNING` con `triggered_by` correcto, y termina en `COMPLETED` con `public_code`. Los cuatro casos del ciclo de vida (`NEW`, `PERSISTING`, `REOPENED`, `RESOLVED` automático) probados uno por uno contra el endpoint real de completado. `PATCH /api/vulnerabilities/{id}/status` confirmado escribiendo sobre `asset_vulnerabilities` sin tocar la fila de `vulnerabilities_audit` correspondiente. Kanban y Dashboard, comparados antes/después vía Playwright, muestran exactamente los mismos datos. Se migraron (backfill) las 9 vulnerabilidades sembradas a `AssetVulnerability`, uno a uno, para no perder el estado ya cargado.

## 11.9 Comparación entre escaneos (Etapa 5)

**Problema.** Con el historial ya trazable (§11.8), faltaba responder la pregunta que motivó la propuesta original: dado dos escaneos, ¿qué cambió entre uno y otro? Qué vulnerabilidades son nuevas, cuáles siguen igual, cuáles se resolvieron, y si alguna que persiste cambió de severidad.

**Alcance decidido.** De las dos etapas que quedaban en la propuesta (`Propuesta_Trazabilidad_Escaneos_GEF_Secure.md`), se implementó esta (Etapa 5) y se dejó explícitamente afuera la Etapa 6 (log de auditoría separado) — la propia propuesta la planteaba como "evaluar si hace falta", condicionada a una pregunta de auditoría concreta que `Scan` + `AssetVulnerability` no pudieran responder, y esa necesidad todavía no apareció en uso real.

**Decisión de diseño.** El endpoint nuevo (`GET /api/scan-reports/{id}/compare/{otherId}`, `VulnerabilityAuditService.compareScans`) se construyó sobre `findByScanReport` (§11.4), no sobre la FK `scan_id` cruda de la Etapa 2. Motivo concreto: `scan_id` recién quedó bien poblado para 1 de los 17 escaneos históricos — el resto solo lo tendrá a futuro, cuando el pipeline de n8n encuentre coincidencias reales contra el inventario (limitación ya documentada en §11.6). Reusar `findByScanReport` — ya probado y usado por el propio detalle del Historial — hace que la comparación funcione igual de bien sobre cualquier escaneo, viejo o nuevo, y quede consistente con lo que el usuario ya ve al clicar una fila.

**Solución implementada.** El servicio trae los hallazgos correlacionados de los dos escaneos, arma un mapa por `cveId` de cada uno, y clasifica: nuevo en B (no está en A), resuelto desde A (estaba en A, no en B), persistente (está en ambos), y dentro de los persistentes, cambio de severidad si la prioridad difiere entre la versión de A y la de B. En el Historial, un botón "Comparar escaneos" abre un popup (mismo componente

**Modal** de §11.5) que primero deja elegir dos escaneos del historial ya cargado y, al comparar, reemplaza su propio contenido por las cuatro categorías — cada una en su propia tabla, reutilizando las columnas ya definidas para el detalle de un escaneo puntual — con un botón "Elegir otros" para volver a la selección sin cerrar el popup.

**Verificación.** Confirmado en vivo por API y por Playwright: comparar el primer escaneo global registrado (que detectó las 9 vulnerabilidades sembradas) contra uno posterior — cuya ventana de correlación no llega tan atrás — muestra correctamente las 9 como "resueltas desde A", con los datos completos de cada una en la tabla del popup.

**Ajuste de UX y defecto corregido en **Modal** (post-implementación).** La primera versión mostraba la comparación como una card fija debajo de la tabla del historial — con 17 escaneos cargados, quedaba fuera de la vista sin scrollear, y el usuario no la encontró. Se movió a un botón junto a "Recargar", siempre visible, que abre el popup descrito arriba. Al verificar esa versión con Playwright apareció un defecto real y preexistente en el componente compartido `components/Modal.tsx`: el cuerpo del popup no tenía altura máxima ni scroll propio, así que un contenido más alto que la pantalla (como la tabla de 9 filas de "Resueltas desde A") empujaba el título y el botón de cerrar fuera del viewport, sin ninguna forma de volver a alcanzarlos. Se corrigió acotando el popup a `max-h-[90vh]` con el encabezado fijo y el cuerpo con scroll interno propio — beneficia a los demás popups de la app (Activos, Kanban, Config), no solo a este.

**Código público del escaneo, expuesto en el frontend (post-implementación).** `ScanReport.publicCode` ( `SCN-2026-000019` ) se había agregado en la Etapa 1 (§11.8) pensado explícitamente como identificador legible para el usuario, pero `ScanReportDTO` nunca llegó a incluirlo — existía en la base pero era invisible en toda la aplicación. Se agregó al DTO y se lo expuso en tres lugares: columna "Código" (primera) en la tabla del Historial, en el título del popup de detalle de un escaneo, y en las etiquetas de los selectores del comparador (§11.9) y en el encabezado del resultado de la comparación.

## 11.10 Estado de tests tras estas mejoras

Los cambios de esta sección con lógica de negocio nueva se implementaron con el mismo criterio de TDD real usado en el resto del proyecto: borrado en cascada y restauración selectiva por timestamp (§11.3), la correlación de resultados de escaneo por reporte histórico (§11.4), el ciclo de vida de vulnerabilidades de la trazabilidad de escaneos (§11.8: `ScanService.start/complete`, `VulnerabilityAuditService.applyLifecycle`, y la redirección de `findAll/findByStatus/updateStatus/delete`), y la comparación entre escaneos (§11.9: `VulnerabilityAuditService.compareScans`). Los rediseños de UI (§11.5) y la unificación/backfill de datos de prioridad (§11.6) no agregaron lógica de negocio nueva en el backend y no requirieron tests adicionales. El total de tests backend subió de 36 (cierre de Fase 4) a **68** (0 fallos): `AssetServiceTest` (13 tests, +6: cascade de borrado, restauración selectiva, exclusión de eliminados en listados), `SoftwareComponentServiceTest` (12 tests, +1: borrado lógico individual), `VulnerabilityAuditServiceTest` (25 tests, +14: ventana con límite superior, resolución de objetivo y acotamiento por reporte anterior, ciclo de vida NEW/PERSISTING/REOPENED/RESOLVED con los dos ejes de estado, alcance por componente/entorno/global, borrado con desvinculación de hallazgos, comparación entre escaneos con y sin cambios de severidad) y `ScanServiceTest` (5 tests, nuevo: creación en `RUNNING` con `public_code`, resolución de componente/entorno, completado con métricas, 404). Los catálogos administrables de §11.11 no modificaron este total (ver justificación allí). La auditoría de datos congelados vs. en vivo (§11.12) sí agregó tests — ver el total final actualizado en esa sección (72).

## 11.11 Catálogos administrables en Config (Entornos, Tipos de Host, Ecosistemas)

**Problema.** Con la trazabilidad de escaneos ya cerrada, quedaban tres huecos de administración concretos, encontrados al revisar qué haría falta para operar el sistema de punta a punta sin tocar código ni la base de datos a mano: `EnvironmentController` ya tenía el CRUD completo ( `GET/POST/PUT/DELETE /api/environments` , escritura `ADMIN` -only) pero nunca se le había construido una pantalla — un Entorno nuevo solo se podía crear insertándolo directamente en Postgres; y los "Tipos de host" ( `SERVIDOR/APLICACION/VM/CONTENEDOR/ENDPOINT` ) y los "Ecosistemas" ( `npm/pip/maven/...` ) que ofrecen los formularios de alta de Activos y Componentes estaban hardcodeados como arrays de TypeScript en `Assets.tsx` , sin ninguna entidad de backend detrás — agregar un tipo de host o un ecosistema nuevo requería modificar código y volver a desplegar.

**Alcance decidido.** Se agregaron las tres pestañas (Entornos, Tipos de Host, Ecosistemas) y se descartó explícitamente convertir en catálogo administrable cualquier otro valor fijo del sistema, con motivo puntual en cada caso: los cuatro roles ( `ADMIN/SECURITY_ANALYST/ASSET_OWNER/AUDITOR` ) están atados a `@PreAuthorize` en todos los controllers — un catálogo editable permitiría crear un "rol" sin ningún permiso real detrás, rompiendo el RBAC de la Fase 4 (§6) en silencio; la prioridad de vulnerabilidad y los estados del Kanban están atados a la matriz de riesgo de n8n y a `TRANSICIONES_PERMITIDAS` en el backend, no son datos administrables sino código; el Catálogo de Software (Fase 1, §3) es un espejo autopoblado desde GitHub Advisory, pensado para no curarse a mano; y la criticidad de negocio de un Entorno ( `BAJA/MEDIA/ALTA/CRITICA` ) es un campo del propio formulario de Entorno, no una entidad separada.

**Decisión de diseño: catálogos livianos, sin FK real.** `AssetType` y `Ecosystem` son catálogos de sugerencia para los `<select>` de alta — no se convirtió `assets.asset_type` ni `software_components.ecosystem` en foreign key real hacia ellos. Motivo: son columnas de texto libre desde la Fase 2, sin ninguna fila que dependiera de una FK; migrar a una relación real hubiera implicado tocar `AssetService` / `SoftwareComponentService` /sus DTOs y mappers para un beneficio marginal, ya que el sistema tampoco valida esos valores hoy. Con el catálogo liviano, borrar un tipo o ecosistema del catálogo no tiene ningún efecto en cascada: los activos y componentes ya creados conservan su valor tal cual — la tabla lo renderiza directo desde el string guardado ( `String(v)` , sin ningún `.find()` contra el catálogo vigente) — y solo deja de ofrecerse como opción para altas nuevas.

**Solución implementada.** `AssetTypeController` ( `/api/asset-types` ) y `EcosystemController` ( `/api/ecosystems` ) replican exactamente el patrón liviano ya usado por `EnvironmentController` : sin capa de `Service` , controller directo sobre el repositorio, lectura abierta a cualquier rol autenticado y escritura `@PreAuthorize("hasRole('ADMIN')")` . Gobernanza LOW (catálogo simple) según las instrucciones del proyecto — autonomía completa, sin checkpoints intermedios una vez aprobado el plan. `init/10-config-catalogs.sql` sembró los 5 tipos de host y los 8 ecosistemas que antes vivían hardcodeados en el frontend, para que los formularios no se quedaran sin opciones al migrar. En `Settings.tsx` se agregaron las 3 pestañas nuevas dentro del mismo gate `ADMIN` -only que ya tenía toda la página Config; en `Assets.tsx` los dos arrays hardcodeados se reemplazaron por `assetTypes` / `ecosystems` cargados con `useEffect` desde los endpoints nuevos, mismo patrón ya usado para cargar `environments` .

**Defecto encontrado y corregido durante la verificación.** Al probar los endpoints nuevos contra el contenedor Docker en ejecución, `GET /api/asset-types` y `/api/ecosystems` devolvían 500 ("No static resource") — el contenedor `gef_backend` corriendo era anterior a la compilación que incluía los controllers nuevos (el `./gradlew build` en verde solo valida el JAR, no lo que está desplegado). Se reconstruyó la imagen ( `docker compose build backend` ) y se reinició el contenedor; ambos endpoints respondieron 200 con los datos sembrados.

**Sin tests nuevos (decisión, no omisión).** Igual que `EnvironmentController` — que tampoco tiene `EnvironmentServiceTest` — los dos controllers nuevos son CRUD directo sobre el repositorio sin lógica de negocio propia, por lo que no se agregaron tests dedicados; el total de backend se mantiene en 68 (\$11.10).

**Verificación.** `npm run build` sin errores de TypeScript. En vivo con Playwright, extremo a extremo: creación y edición de un Entorno (criticidad y descripción reflejadas correctamente en la tabla); creación de un Tipo de Host y de un Ecosistema, confirmando que aparecen de inmediato en el `<select>` "Tipo" del alta de Activos y en el `<select>` "Ecosistema" del alta de Componentes, respectivamente; borrado de los tres registros de prueba, confirmando que desaparecen de sus tablas y quedan sin datos de prueba remanentes. Sin errores de consola del navegador en ningún paso.

## 11.12 Auditoría de datos "congelados vs. en vivo": entorno del componente, estado del hallazgo y objetivo del escaneo

Después de cerrar los catálogos de Config, se hizo una revisión deliberada del resto del modelo buscando la misma clase de problema en otros lugares: un valor guardado de forma independiente que debería derivarse siempre de una relación en vivo, y que puede desincronizarse silenciosamente de ella. Aparecieron tres casos, uno de ellos ya activamente divergente en datos reales.

### Caso A — `SoftwareComponent.environment` independiente del entorno de su propio activo.

`SoftwareComponent` conservaba su propia columna `environment_id`, seteable en el formulario de alta/edición de forma **independiente** del `environment_id` de su `Asset` — arrastrada desde la Fase 2, cuando el componente era la única entidad y todavía no existía `Asset`. Con `asset_id` ya obligatorio desde la \$11.3, ese campo quedó puramente duplicado con `asset.environment_id`, sin ningún mecanismo que los mantuviera sincronizados. Se confirmó divergencia real contra la base de desarrollo: el componente "Axios HTTP Client" tenía `environment_id=2` mientras su propio activo tenía `environment_id=1`. No era solo un dato inconsistente en pantalla — ese campo es el que usaba `VulnerabilityAuditService.applyLifecycle` para resolver el alcance de un escaneo por ENTORNO, y el que usaba la query nativa `findByEnvironmentScanWindow` para el resultado de escaneo en vivo; un componente con el entorno "equivocado" podía hacer que un escaneo de "Testing" resolviera mal qué vulnerabilidades pertenecían a ese entorno.

**Solución.** Se eliminó `environment_id` de `software_components` por completo (`init/11-drop-component-environment.sql`) — el entorno de un componente pasa a derivarse siempre de `component.getAsset().getEnvironment()`, nunca de un campo propio. La migración se topó con una vista muerta (`view_vulnerability_details`, de `init/01-create-tables.sql`, sin ninguna referencia en el código de la aplicación y ya stale desde la Etapa 3/4) que dependía de la columna — se dropeó en vez de recrearse. El chequeo de duplicados de `SoftwareComponentService` (mismo software+versión en el mismo entorno) se re-scopeó para resolver el entorno a través del activo (`findBySoftwareAndVersionAndAsset_Environment_IdAndDeletedAtIsNull`, nuevo método de repositorio), igual que `AssetVulnerabilityRepository.findBySoftwareComponent_Environment_IdAndDetectionStatus` → `..._Asset_Environment_IdAndDetectionStatus` y la query nativa `findByEnvironmentScanWindow` (join agregado contra `assets`). En el frontend, el `<select>` de Entorno del formulario de Componente se reemplazó por un campo de solo lectura con el valor heredado del activo elegido.

**Verificación.** El componente "Axios HTTP Client" pasó a reportar correctamente el entorno de su activo (antes divergente) vía `GET /api/software-components`. El chequeo de duplicados sigue funcionando (409 vía el entorno del activo; sin falso positivo entre entornos distintos, confirmado creando el mismo software+versión en un entorno diferente). En vivo con Playwright: el modal de Componente ya no ofrece seleccionar entorno, lo muestra heredado del host.

**Caso B — `status / decision / assignedTo / resolvedAt` de un hallazgo puntual, mostrados como si fueran el estado actual.** Se reprodujo en vivo, no era hipotético: se cambió el CVE-2024-4317 a `RESUELTA` vía el endpoint que usa el Kanban (que desde la Etapa 4 escribe sobre `AssetVulnerability`) y se confirmó contra la base que el hallazgo original en `vulnerabilities_audit` seguía en `EN_ANALISIS` — el Kanban mostraba "RESUELTA" (correcto, lee `AssetVulnerability`), pero el popup de detalle del Historial, el comparador de escaneos y el resultado de escaneo en vivo armaban su respuesta con `VulnerabilityAuditMapper.toResponse(VulnerabilityAudit entity)`, que copiaba `status / decision / assignedTo / resolvedAt` directo del hallazgo — congelados desde el INSERT y nunca reescritos (se confirmó que el overload `applyStatusUpdate(StatusUpdate, VulnerabilityAudit)` que en teoría los actualizaría está muerto, sin ningún caller desde la redirección de la Etapa 4). Mismo CVE, dos pantallas, dos estados distintos.

Es distinto de `priority / cvss`, que sí son intencionalmente un snapshot histórico (es justamente lo que usa `compareScans` para detectar cambios de severidad entre corridas, §11.9) — `status / decision / assignedTo / resolvedAt` no tienen ese propósito.

**Solución.** Se agregaron cuatro métodos `default` al mapper (`resolveCurrentStatus/Decision/AssignedTo/ResolvedAt`) que prefieren el `AssetVulnerability` vinculado al hallazgo cuando existe, con fallback al valor propio del hallazgo solo para los legacy nunca vinculados (`assetVulnerability == null`, hallazgos sembrados/backfillados antes de la Etapa 3). `cveId`, `cvss`, `exploited`, `priority`, `ghsaId`, `detectedAt` no se tocaron — siguen viniendo del hallazgo, snapshot intencional.

**Verificación.** Se repitió el experimento que confirmó el bug: el mismo CVE marcado "RESUELTA" vía Kanban ahora aparece "RESUELTA" también en `GET /api/scan-reports/{id}/vulnerabilities` (Historial). Cambio de prueba revertido al terminar.

**Caso C — `findByScanReport()` resolvía el objetivo por nombre aunque el reporte ya tuviera el FK real.** `findByScanReport()` (Historial → detalle de un escaneo puntual, y por extensión `compareScans`) resolvía el componente/entorno del escaneo con `softwareComponentRepository.findFirstByName(report.getTargetName())` / `environmentRepository.findFirstByName(...)` — una búsqueda por nombre — incluso para escaneos que ya tenían el FK real (`report.getSoftwareComponent() / report.getEnvironment()`), poblado desde la Etapa 1, §11.8). Se confirmó contra la base que 2 de los 5 escaneos por ACTIVO ya tenían el FK y aun así el código los ignoraba. Riesgo concreto de `findFirstByName`: no filtra por entorno (la Fase 2 permite explícitamente dos activos homónimos en entornos distintos, §4.2), y se rompe si el activo/entorno se renombra después del escaneo, porque `targetName` queda congelado con el nombre viejo.

**Alcance de la corrección.** Solo la resolución de "a qué componente/entorno pertenece este reporte" — la ventana de correlación con "el escaneo anterior del mismo objetivo" se dejó como está, todavía por nombre, porque corregirla del todo requeriría backfillar el FK en los escaneos históricos que nunca lo tuvieron (fuera de alcance, mismo criterio que la limitación ya documentada en §11.9 sobre `scan_id`). Se documenta como decisión, no como omisión.

**Solución.** `findByScanReport()` ahora usa `report.getSoftwareComponent() / report.getEnvironment()` directamente cuando existen, y solo cae a la búsqueda por nombre para escaneos históricos que nunca tuvieron el FK poblado.

**Verificación.** Se renombró temporalmente el componente "Axios HTTP Client" (dueño de un escaneo con el FK ya poblado) y se confirmó que `GET /api/scan-reports/{id}/vulnerabilities` seguía devolviendo su hallazgo

sin problema — con el código anterior, ese rename habría roto la búsqueda por nombre y devuelto una lista vacía. Nombre revertido al terminar.

**Tests.** Primer test de un mapper en este proyecto (`VulnerabilityAuditMapperTest`, instanciando `VulnerabilityAuditMapperImpl` directo, sin contexto de Spring) — se justificó porque el fallback agregado es lógica condicional real, no solo renombrado declarativo de campos: reproduce exactamente el Caso B (hallazgo con `status="EN_ANALISIS"` vinculado a un `AssetVulnerability` con `triageStatus="RESUELTA"` → el DTO debe exponer "RESUELTA"), y su triangulación (sin vínculo, cae al valor propio). Dos tests nuevos en `VulnerabilityAuditServiceTest` para el Caso C, verificando con `verify(..., never())` que `findFirstByName` no se invoca cuando el reporte ya trae el FK. El total de tests backend subió de 68 a **72** (0 fallos).

### 11.13 Correlación real de escaneos con GitHub Advisory (filtrado por inventario)

**Problema.** El nodo `Fetch_GitHub_Advisories` del pipeline de n8n llamaba `GET https://api.github.com/advisories` **sin ningún parámetro** — traía un lote genérico de advisories recientes, de cualquier ecosistema, sin relación con el inventario real (`software_components`). Esa misma llamada alimentaba dos cosas a la vez: el matching de cada escaneo (`Check_Internal_Assets`) y el catálogo de autocomplete de Activos (`Sync_Software_Catalog`, §3). Confirmado contra la base antes del cambio: **0 escaneos, de las 16+ ejecuciones de esta sesión, insertaron jamás un hallazgo real**, y el catálogo tenía 22 paquetes ajenos al inventario (ninguno de los 6 componentes semilla). Era la limitación ya documentada en §10 ("Pipeline de n8n sin correlación real de advisories"), pendiente desde el cierre de esa sección para una sesión dedicada.

**Decisión de diseño.** En vez de consultar una vez sin filtro, se agregó un nodo nuevo `Get_Installed_Packages` (Postgres) que resuelve los pares (`software, ecosystem`) distintos realmente instalados, con el mismo alcance (`filtro_entorno / filtro_activo`) que ya usaba `Check_Internal_Assets`, y filtrados contra el propio catálogo de Ecosistemas administrable (§11.11) para excluir valores que GitHub no reconoce (ver más abajo). `Fetch_GitHub_Advisories` pasa a recibir esa lista como entrada y consulta una vez por paquete (`? ecosystem=X&affects=Y`, parámetros reales soportados por la API de GitHub Advisory) — n8n ejecuta el nodo una vez por ítem de entrada automáticamente, sin necesidad de un nodo de loop explícito. Todo lo que sigue después (`Extract_Advisory_Items`, `Deduplicate_Advisories`, `Sync_Software_Catalog`, `Check_Internal_Assets`, `Risk_Assessment_Engine`, `Persist_Audit_Findings`, ...) no se tocó — ya estaba bien construido para procesar coincidencias reales, solo nunca las recibía.

**Regresión encontrada en el camino.** Al revisar el SQL embebido en el pipeline (nunca cubierto por el grep sobre el código Java/TS), `Check_Internal_Assets` seguía con `JOIN environments e ON sc.environment_id = e.id` — la columna que se había eliminado en §11.12 Caso A sin revisar el workflow de n8n. Cualquier escaneo real fallaba en ese nodo desde esa migración (confirmado: `system_errors` ya tenía una fila con exactamente ese error, de una ejecución anterior de esta misma sesión). Se corrigió el JOIN para pasar por `assets (a.environment_id)`, igual que el resto de las consultas ya corregidas en §11.12.

**Ecosistemas no reconocidos por GitHub — docker como caso concreto.** El componente semilla `postgres` está cargado con ecosistema `docker`, que no es un valor válido para GitHub Advisory (`GET /advisories? ecosystem=docker` devuelve 422: "Invalid input: `docker` is not a possible value"). La solución fue filtrar `Get_Installed_Packages` para que solo considere ecosistemas presentes en el catálogo administrable de Ecosistemas (`public.ecosystems`, §11.11) — como `docker` nunca estuvo ahí, queda excluido en el origen, en vez de dejar que la llamada a GitHub falle después. Un intento intermedio de manejar el 422 con `alwaysOutputData` en `Fetch_GitHub_Advisories` generó un ítem vacío que rompía `Deduplicate_Advisories` (comparación estricta por `ghsa_id` faltante) — se revirtió en favor del filtro en el origen, más simple y sin

efectos secundarios sobre el resto del pipeline. `cargo` (nuestro nombre para el ecosistema de Rust) sí se traduce, porque GitHub lo llama `rust`.

### Verificación en vivo — todos los tipos de escaneo, con hallazgos reales:

| Tipo    | Caso                                                                                   | Resultado                                                                                                                  |
|---------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| GLOBAL  | Todo el inventario                                                                     | 31 hallazgos reales, incluido <code>GHSA-rv95-896h-c2vc</code> (express) — el mismo GHSA que ya vivía en los datos semilla |
| ENTORNO | Desarrollo (react, express, axios)                                                     | 27 hallazgos reales, 0 errores                                                                                             |
| ACTIVO  | Componente normal (express)                                                            | 3 hallazgos reales                                                                                                         |
| ACTIVO  | Caso límite ( <code>postgres</code> / <code>docker</code> , sin ecosistema reconocido) | Completa <code>COMPLETED</code> con 0 hallazgos falsos, sin quedar trabado en <code>RUNNING</code>                         |

El catálogo de autocomplete ( `software_catalog` ) pasó a poblarse con paquetes reales del inventario ( `express` , `axios` , `react` , `lodash` , todos `npm` ) como efecto secundario directo del mismo fetch ya filtrado — cierra también la pregunta abierta sobre si el ecosistema debía autocompletarse al elegir software del catálogo: el mecanismo ( `handleCatalogSelect` bloqueando el `<select>` de ecosistema) ya existía en `Assets.tsx` , solo le faltaban datos relevantes para ser útil en la práctica.

**Investigación del disparador diario ( `Scan_Scheduler` , cron a las 9am).** Verificar este caso requirió una investigación más profunda que los demás, documentada porque el camino reveló información real sobre la infraestructura:

- Un primer intento de simular el cron (cambiar el intervalo a "cada 1 minuto" vía la API de n8n) nunca disparó ejecuciones — porque cada reinicio de n8n (necesario para que cualquier cambio de nodo tome efecto, ver §11.8) hace que el script de arranque re-importe el workflow **desde el archivo del repositorio**, revirtiendo silenciosamente cualquier cambio aplicado solo vía API que no se hubiera sincronizado también al archivo.
- Corregido eso, apareció un problema real y más serio: activar logging en modo `debug` para diagnosticar rompió el script de arranque ( `n8n/bootstrap.sh` ) — su función `publish_all()` filtra la salida de `n8n list:workflow` buscando líneas con el separador `|` para extraer IDs de workflow; en modo debug, el CLI también imprime líneas de log con formato `TIMESTAMP | debug | ...` , que también contienen `|` . El script tomó una marca de tiempo como si fuera un ID de workflow, `publish:workflow` falló ( `Workflow "<timestamp>" not found` ), y el workflow quedó **desactivado silenciosamente** — rompiendo webhooks y cron por igual, sin ningún error visible para el usuario del sistema. No es un problema del cron en sí ni de este rediseño: en el nivel de log normal de producción (sin `debug` ), el script publica correctamente — confirmado con un reinicio limpio posterior.
- Con logging en `debug` (ya revertido) se pudo confirmar el mecanismo interno real de n8n: el nodo registra el cron correctamente ( `Registered cron for workflow` , `instanceRole: leader` ) y lo ejecuta exactamente en el segundo programado ( `Executing cron for workflow` , dos disparos consecutivos y puntuales), insertando hallazgos reales nuevos en la base cada vez.
- Estado final: `docker-compose.yml` sin el `N8N_LOG_LEVEL=debug` de diagnóstico, `Scan_Scheduler` de vuelta a `triggerAtHour: 9` (producción) tanto en la instancia viva como en el archivo del repositorio, cron registrado y confirmado ( `"cron": "43 0 9 * * *", "instanceRole": "leader"` ).

**Segundo defecto operativo encontrado en el propio proceso de despliegue.** Al sincronizar el archivo del repositorio con la versión corregida del workflow, un primer script de sincronización omitió el campo `id` del JSON exportado — el script de arranque de n8n, al reimportar un workflow sin `id` , crea uno **nuevo** en vez de

actualizar el existente por coincidencia de ID. Esto generó un workflow duplicado activo con el mismo webhook, y el tráfico real quedó yendo al duplicado (con la configuración vieja) en vez del que se estaba corrigiendo. Se detectó comparando las ejecuciones reales de n8n contra ambos IDs, se borró el duplicado, y se corrigió el script de sincronización para incluir siempre el `id`.

**Sin tests nuevos.** Cambio exclusivamente sobre el workflow de n8n (JSON de nodos/conexiones) — mismo criterio que el resto de la trazabilidad de escaneos: sin build ni test runner propio en este repositorio para ese artefacto (documentado en §10). El total de tests backend se mantiene en 72.

## 11.14 Detalle de un escaneo: filtrar por `scan_id` real, no por ventana de tiempo

**Problema.** Reportado en vivo por el usuario: el detalle de `SCN-2026-000033` (ACTIVO, Express API Gateway) mostraba 14 vulnerabilidades, pero la propia fila del Historial indicaba que ese escaneo detectó solo 3. Causa: `findByScanReport()` (§11.4) nunca filtraba por el escaneo exacto — armaba una ventana de tiempo ("desde el escaneo anterior del mismo objetivo hasta este") y traía todo lo que cayera ahí para ese activo. Como `SCN-2026-000033` era el único escaneo ACTIVO corrido alguna vez sobre Express, no había "escaneo anterior" que acotara el límite inferior: la ventana quedaba abierta desde el principio de los tiempos y arrastraba, además de los 3 hallazgos reales de esa corrida, los de otros escaneos GLOBAL/ENTORNO que también habían tocado a Express en la misma sesión de pruebas. Confirmado contra la base: de las 14 filas devueltas, solo 3 tenían `scan_id=33`.

Esta heurística de ventana era la única opción viable cuando se implementó (§11.4/§11.9), porque `scan_id` recién se empezó a completar de forma confiable a partir de la Etapa 2 de la trazabilidad de escaneos, y hasta la corrección de §11.13 el pipeline casi nunca insertaba hallazgos reales de todos modos — el FK rara vez tenía algo que aportar. Con ambas cosas resueltas, la heurística de ventana quedó obsoleta para este método: el FK real es ahora la fuente de verdad correcta, y usar la ventana en su lugar podía mezclar resultados de escaneos distintos que compartían objetivo y cercanía temporal.

**Solución implementada.** `findByScanReport(reportId)` pasa a devolver exactamente `repository.findByScanReport_Id(reportId)` — sin ventana de tiempo, sin resolución de activo/entorno por nombre ni por FK (la lógica de Caso C de §11.12 quedó superada y se eliminó junto con el resto). Un escaneo que genuinamente no encontró nada muestra 0, en vez de completar el hueco con resultados de otro escaneo. No se tocó `findScanResult()` / `GET /api/vulnerabilities/scan-result` (la vista de "resultado de escaneo en vivo" mientras una corrida recién disparada todavía no tiene hallazgos persistidos vía `scan_id` para consultar) ni las queries de ventana que ese método usa — siguen siendo la única opción viable para ese caso de uso distinto.

**Código muerto eliminado como consecuencia** (sin otros callers en el proyecto, confirmado por grep):

```
EnvironmentRepository.findFirstByName (y el campo environmentRepository completo en
VulnerabilityAuditService, que era su único uso en esa clase),
SoftwareComponentRepository.findFirstByName,
ScanReportRepository.findFirstByTargetTypeAndTargetNameAndExecutedAtLessThanOrderByExecutedAtDesc.
```

**Tests.** Se reemplazaron los 5 tests existentes de `findByScanReport` (ACTIVO/ENTORNO/GLOBAL por heurística, más los 2 de preferencia de FK de §11.12, todos sobre el mecanismo viejo) por 2 nuevos: uno que confirma que el método devuelve exactamente lo vinculado por `scan_id`, y otro que reproduce directamente el bug reportado verificando que ninguno de los tres mecanismos de ventana de tiempo se invoca. Los 3 tests de `compareScans()` (que llama `findByScanReport` internamente) se actualizaron para mockear `findByScanReport_Id` en vez de la ventana, sin cambios en su propia lógica de armado de mapas/diffs. El total

de tests backend pasó de 72 a **69** (0 fallos) — baja porque 5 tests de un mecanismo eliminado se reemplazan por 2 más simples y directos, no por pérdida de cobertura.

**Verificación.** En vivo: `GET /api/scan-reports/33/vulnerabilities` pasó de devolver 14 a devolver exactamente 3, con timestamps que corresponden precisamente a la ejecución de ese escaneo. El comparador de escaneos (§11.9) probado contra dos escaneos GLOBAL reales (27 y 34, casi idénticos) devolvió 31 persistentes / 0 nuevas / 0 resueltas / 1 cambio de severidad — coherente. El escaneo del componente `postgres / docker` (el caso límite de §11.13, sin ecosistema reconocido por GitHub) pasó a mostrar 0 vulnerabilidades en su detalle en vez de heredar resultados de otro escaneo — mismo tipo de corrección, confirmada también para ese caso.

## 11.15 Drill-down por activo en el detalle de escaneos GLOBAL/ENTORNO

**Problema.** El detalle de un escaneo (Historial, y el resultado recién disparado en la pestaña "Escaneos") mostraba, para ACTIVO, la tabla detallada de vulnerabilidades (CVE/GHSA, CVSS, prioridad, estado, fecha de detección). Para GLOBAL y ENTORNO, en cambio, solo se veía un resumen agrupado por activo (nombre, cantidad, severidad más alta) — `groupByAsset()` armaba ese resumen a partir de la lista plana de hallazgos, pero descartaba las filas originales, así que no había forma de entrar al detalle de un activo puntual dentro de un escaneo con varios.

**Solución implementada.** Nuevo componente `GroupedVulnerabilityResult` (`pages/Scans.tsx`): para ACTIVO muestra el detalle directo sin cambios; para GLOBAL/ENTORNO muestra el resumen agrupado con las filas ahora clickeables (reusando `onRowClick / keyField`, ya soportado por el componente `Table` genérico) — al hacer click en un activo, filtra en el cliente la misma lista de hallazgos ya cargada y muestra su detalle con la tabla `vulnColumns`, con un botón "← Volver al resumen" para regresar sin cerrar el popup. Sin cambios de backend: `VulnerabilityAuditDTO.Response` ya trae `asset / assetId` por hallazgo, el drill-down es filtrado sobre datos que ya habían llegado al cliente. El componente se usa en los dos lugares que compartían el mismo patrón — el resultado en vivo de la pestaña "Escaneos" y el popup de detalle del Historial — para no dejar uno corregido y el otro no.

**Verificación.** En vivo con Playwright, en ambos lugares: en el Historial, abrí `SCN-2026-000034` (GLOBAL), confirmé el resumen agrupado (Lodash Utils/4/CRITICAL, Servidor Demo/24/HIGH, Express API Gateway/3/MEDIUM), hice click en "Lodash Utils" y vi las 4 CVE exactas de ese activo en ese escaneo, y volví al resumen sin cerrar el modal. En la pestaña "Escaneos", disparé un escaneo ENTORNO real sobre "Desarrollo" y confirmé el mismo comportamiento en el resultado recién completado. Sin errores de consola. El caso ACTIVO confirmado sin cambios (detalle directo, sin resumen intermedio).

**Sin tests nuevos ni de backend.** Cambio de frontend puro, sin test runner configurado para ese lado del proyecto (mismo criterio ya documentado en el resto del informe para cambios de UI).

## 11.16 Flujo inyector: carga automática de un activo de prueba para probar escaneos eficientemente

**Problema.** Probar el pipeline de escaneos (§11.13) de punta a punta dependía de tener, a mano, activos con software real e instalado que efectivamente tuviera vulnerabilidades publicadas — algo que no siempre estaba disponible en el inventario de desarrollo sin cargarlo a mano. El proyecto ya tenía un intento de resolver esto — el "FLUJO INYECTOR" de n8n (`Manual_Trigger` → `Fetch_Global_Advisories` → `Random_Asset_Selector` → `Upsert_Test_Asset` → `Notify_Dev_Slack1`) — pero estaba roto en varios frentes a la vez: `Upsert_Test_Asset` nunca seteaba `asset_id` (obligatorio desde la Fase 2, §4), su `ON_CONFLICT` apuntaba a una constraint que ya

no existe, y el flujo entero dejó de andar cuando se eliminó `software_components.environment_id` en §11.12 Caso A (decisión explícita en ese momento de posponerlo para una sesión dedicada). Además, `Manual_Trigger` solo es disparable desde el editor de n8n — no había forma de activarlo desde la aplicación.

**Decisión de diseño — variedad de severidad, no solo activos de CISA KEV.** El diseño original evaluado para el rediseño sourceaba candidatos desde CISA KEV (vulnerabilidades con explotación activa confirmada). Se descartó ese enfoque a pedido explícito: KEV sesga casi todo a `HIGH / CRITICAL` por definición, y el objetivo era simular un inventario realista — con vulnerabilidades de severidad variada (baja/media/alta/crítica), no un activo todo-crítico. La solución final consulta GitHub Advisory **bucketeado por severidad** (`GET /advisories?severity={low|medium|high|critical}`), reutilizando el mecanismo "un ítem de entrada = una llamada HTTP" ya usado en `Get_Installed_Packages → Fetch_GitHub_Advisories` (§11.13), y arma el host con 1 candidato de cada uno de los 4 buckets más un 5° elegido al azar entre el resto — todos CVE/GHSA reales, con `vulnerable_version_range` real, que el pipeline de escaneo puede detectar de verdad. La prioridad final que termina mostrando el Kanban no la copia de la severidad de GitHub: la calcula igual que para cualquier otro hallazgo el `Risk_Assessment_Engine` del pipeline principal, en base al CVSS — bucketear por severidad en el origen solo asegura variedad real en esa entrada, no un resultado artificialmente prefabricado.

**Backend.** Nuevo método `N8nWebhookService.triggerAssetInjection()` — mismo patrón que `triggerScanForAsset/Environment/GlobalScan`, un `RestClient.post()` sin body contra una nueva URL de webhook (`n8n.inject-webhook.url`, `docker-compose.yml / application.properties`, análoga a `n8n.webhook.url`). Nuevo endpoint `POST /api/assets/inject-test-data` en `AssetController`, `@PreAuthorize("hasRole('ADMIN')")` — a diferencia del resto de escritura de Activos (`ADMIN + SECURITY_ANALYST`), este quedó ADMIN-only a pedido explícito. Responde `202 Accepted` de inmediato; no hay estado tipo `ScanReport` que trackear porque la inyección es rápida y no tiene ciclo de vida `RUNNING/COMPLETED`.

**Frontend.** Botón nuevo "Inyectar Activo" en `Assets.tsx`, gateado por `isAdmin = user?.role === 'ADMIN'` (distinto de `canWrite`, que también incluye `SECURITY_ANALYST`), visible solo en la pestaña "Vigentes". Al hacer click llama `assetApi.injectTestData()`, muestra un toast "Inyectando...", espera ~4s (tiempo estimado de la cadena completa: 4 llamadas a GitHub por severidad + 2 INSERTs) y recarga la tabla de Activos y Componentes con un toast de éxito.

**Rediseño del flujo en n8n.** Se reemplazaron los 5 nodos rotos por una cadena de 10 nodos nuevos: `Webhook_Inject_Asset` (`POST /webhook/inject-test-asset`, dispara desde el backend en vez de requerir el editor) → `Prepare_Severities` (arma los 4 ítems `low / medium / high / critical`) → `Fetch_Advisories_By_Severity` (una llamada HTTP por severidad) → `Extract_And_Tag` (`SplitOut`, mismo patrón que `Extract_Advisory_Items` del pipeline principal, conservando la severidad de origen) → `Build_Candidate_Pool` (filtra a los ecosistemas del catálogo administrable, §11.11, y deduplica por software+ecosistema) → `Pick_5_Balanced` (1 candidato al azar por severidad + 1 extra; entorno del host asignado al azar entre los 3 existentes) → `Insert_Host` (INSERT directo en `assets`, `RETURNING id`) → en paralelo, `Split_Components → Insert_Components` (INSERT directo en `software_components`, uno por cada uno de los 5) y `Notify_Injection_Complete` (Slack, mismo canal que ya usaba el flujo original). Igual que el diseño original, la inserción es SQL directo contra Postgres, no vía la API REST del backend — evita tener que emitir un JWT de servicio para n8n, complejidad que no aporta nada para una herramienta exclusivamente de datos de prueba; como consecuencia aceptada, los componentes inyectados quedan `catalogued=false / purl=null` (no pasan por `SoftwareComponentService.resolveCatalogAndPurl`), sin impacto real porque el pipeline de escaneo los encuentra por `Get_Installed_Packages`, que no usa esos campos. Workflow actualizado vía la API REST de n8n (`PATCH /rest/workflows/{id}`), sincronizado al archivo del repositorio **incluyendo el campo `id`**

(la lección operativa de §11.13 sobre workflows duplicados) y verificado tras reiniciar el contenedor: un solo workflow, sin duplicados, `active: true`, 42 nodos.

**Defecto encontrado durante la verificación — imágenes Docker desactualizadas.** La primera pasada de verificación en vivo falló en dos frentes a la vez: el botón "Inyectar Activo" no aparecía para ningún rol (ni siquiera ADMIN), y al probar el endpoint directo devolvía `500 Request method 'POST' is not supported`. La causa no era el código — `./gradlew test` y `vite build` ya habían pasado en verde antes, como en el resto de esta sesión — sino que esa verificación local nunca se traduce sola en los contenedores corriendo: `gef_frontend` y `gef_backend` seguían sirviendo las imágenes Docker construidas *antes* de estos cambios de código, con Spring Security devolviendo 401 (antes de resolver la ruta) para las llamadas sin token, lo cual enmascaraba que el handler real ni siquiera existía en el JAR corriendo. Confirmado inspeccionando el bundle servido por nginx (`Inyectar Activo` ausente del JS empaquetado) y la fecha de creación de la imagen del backend. Se corrigió con `docker compose build frontend backend` + `docker compose up -d frontend backend`, y se repitió la verificación completa desde cero. Queda documentado como recordatorio operativo: en este proyecto, verificar en vivo contra los contenedores requiere reconstruirlos explícitamente después de cambios de código — un `gradlew test` / `vite build` en verde no alcanza por sí solo.

### Verificación en vivo.

| Paso                                            | Resultado                                                                                                                                                                                                                                             |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Webhook disparado directo ( <code>curl</code> ) | 1 fila nueva en <code>assets</code> , exactamente 5 en <code>software_components</code> , todas con <code>software</code> / <code>ecosystem</code> / <code>version</code> reales                                                                      |
| Variedad de severidad                           | Confirmada en 2 corridas independientes: severidades <code>low</code> / <code>medium</code> / <code>high</code> / <code>critical</code> representadas entre los 5 candidatos, nunca las 5 en <code>CRITICAL</code>                                    |
| Escaneo GLOBAL real sobre el host inyectado     | 27 hallazgos detectados y auditados por el pipeline (§11.13) sobre 4 de los 5 componentes, con las 4 severidades presentes en los resultados y prioridades calculadas por <code>Risk_Assessment_Engine</code> (no copiadas de la severidad de origen) |
| Playwright — gateo por rol                      | Botón visible solo para <code>fede.frankenberger</code> (ADMIN); ausente para <code>analyst.demo</code> (SECURITY_ANALYST) y <code>owner.demo</code> (ASSET_OWNER)                                                                                    |
| Playwright — flujo de click completo            | Click → toast "Inyectando..." → toast de éxito → nueva fila en la tabla de Activos con 5 componentes, entorno asignado al azar (Producción/Testing/Desarrollo, los 3 observados en corridas distintas)                                                |
| Limpieza                                        | Los hosts de prueba creados durante la verificación se borraron con el borrado lógico en cascada ya existente de Activos (§4.3); confirmado que no queda ninguno activo                                                                               |

**Sin tests nuevos de backend.** Endpoint de disparo simple sin lógica propia (delega todo a n8n), mismo criterio que `EnvironmentController` /el resto de triggers de `scanApi`. Sin tests de n8n (JSON de nodos) ni de frontend (cambio de UI puro), mismo criterio documentado en el resto de esta sección. El total de tests backend se mantiene en **69**.

## 11.17 Escanear un Activo (host) completo, con todos sus componentes en una sola acción

**Problema.** Al usar el flujo inyector (§11.16) para probar un host con 5 componentes, el usuario reportó que ese host no aparecía como opción al ir a escanearlo, y que en su lugar aparecían sus 5 software sueltos, uno por uno. Causa: "Centro de Escaneos" solo tenía 3 modos — **Componente** ( `targetType='ACTIVO'`, nombre heredado de antes de la Fase 2, cuando `SoftwareComponent` se llamaba "Asset"; dispara sobre 1 `SoftwareComponent` puntual), **Entorno** y **Global** — nunca existió un nivel "todos los componentes de este Asset puntual". Un host con 1 solo componente no hacía notar el problema; con 5, sí.

**El gap era solo de la app, no de n8n.** Investigando el pipeline se encontró que el nodo `Check_Internal_Assets` ya hacía el matching contra `a.name = filtro_activo OR sc.name = filtro_activo` (`workflows/Pipeline de Gestión Automatizada de Vulnerabilidades (VMP).json`) — es decir, ya sabía filtrar por nombre de Asset (host), no solo por nombre de Componente. Nunca se había armado el lado de la app que disparara pasando el nombre del Asset en vez del nombre del Componente. Resultado: la feature terminó siendo más chica de lo que parecía — **cero cambios en n8n.**

**Nuevo modo de escaneo "Activo".** Se agregó `targetType='HOST'` en `scan_reports` (sin reusar `'ACTIVO'`, que ya significa Componente históricamente — reusarlo habría roto el filtrado del Historial y el comparador sobre escaneos ya existentes). En la UI, la tira de botones de "Nuevo escaneo" pasó a ser **Componente / Activo / Entorno / Global.**

**Evitar el bug de homónimos.** El schema permite explícitamente 2 Activos con el mismo nombre en entornos distintos (§4.2, `UNIQUE(name, environment_id)`). Disparar solo con `activo_name` y `entorno='TODOS'` (como hace `triggerScanForAsset` para Componente) podía traer también los componentes de otro host homónimo en otro entorno. Se evitó con un método nuevo en `N8nWebhookService` (`triggerScanForAssetHost`) que manda también el entorno real del Asset — como `Check_Internal_Assets` ya filtra por `entorno` Y `activo` a la vez, alcanzó con dejar de mandar `'TODOS'`. La combinación `(a.name, e.name)` es única por la propia constraint de la tabla, sin ambigüedad posible. Verificado creando 2 Activos "Homónimo Test" en entornos distintos: escanear uno nunca trajo hallazgos del otro (0 filas cruzadas, confirmado contra la base).

**FK real, no solo nombre.** Coherente con el criterio ya aplicado en §11.12/§11.14 (preferir FK vivo sobre matching por nombre), se agregó una columna `asset_id` nullable a `scan_reports` (`init/12-scan-report-asset-fk.sql`, aplicada a mano contra el contenedor corriendo), mismo patrón que ya existía para `software_component_id / environment_id`. `ScanService.start()` ganó un 5º parámetro para resolverla.

**Mostrar qué componente es cada hallazgo.** `VulnerabilityAuditDTO.Response.asset` ya mostraba el nombre del `host` cuando el componente tenía uno asignado (`resolveDisplayName`, usado por el agrupado de §11.15) — perfecto para Entorno/Global, pero inútil para un escaneo de Activo con 5 componentes distintos, porque los 5 hallazgos mostrarían el mismo nombre de host sin poder distinguir a qué software pertenece cada CVE. Se agregó un campo nuevo y aditivo `componentName` al DTO (`entity.getSoftwareComponent().getName()`, sin tocar `resolveDisplayName`), y la tabla de resultado en modo Activo agrega una columna "Componente" al frente — el componente `GroupedVulnerabilityResult` (§11.15) ya usado por Historial y el resultado en vivo la incorpora automáticamente en ambos lugares, sin duplicar código.

**Backend.** Nuevo endpoint `POST /api/assets/{id}/scan` (`ADMIN + SECURITY_ANALYST`, mismo gate que Componente/Entorno) en `AssetController`; nueva query nativa `findByAssetScanWindow` (mismo patrón que `findByEnvironmentScanWindow` pero `WHERE sc.asset_id = :assetId`); `VulnerabilityAuditService.findScanResult / GET /api/vulnerabilities/scan-result` ganaron un parámetro `hostAssetId`. Sin cambios en `ScanReportRepository.search()` (ya filtraba `target_type / target_name` de forma genérica por string) ni en `applyLifecycle()` (ya opera por `scan_id` real desde §11.14, agnóstico al tipo de escaneo).

**Frontend.** `Scans.tsx`: nuevo modo "Activo" con su propio `<select>` de Activos (vía `assetApi.getAll()`, ya existente), disparo vía nuevo `assetApi.triggerScan(id)`, y filtro nuevo equivalente en la pestaña Historial.

**Verificación en vivo.**

| Paso                                                                    | Resultado                                                                                                                                                            |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST /api/assets/{id}/scan sobre el host inyectado                      | ScanReport resultante con asset_id poblado (FK real, no solo nombre)                                                                                                 |
| GET /api/vulnerabilities/scan-result?hostAssetId=X                      | Hallazgos de 4 componentes distintos del mismo host ( contao/contao , traefik , meshcentral , docx4j-core ), cada uno con su componentName propio                    |
| GET /api/scan-reports/{id}/vulnerabilities (detalle exacto por scan_id) | Mismos 14 hallazgos, misma variedad de componentes — confirma que la corrección de §11.14 (FK real, no ventana) funciona igual para escaneos por Activo              |
| Homónimos                                                               | 2 Activos "Homonimo Test" en entornos distintos; escanear uno ( sqlparse ) nunca trajo el componente del otro ( vm2 ) — 0 filas cruzadas                             |
| Playwright — flujo completo                                             | Pestaña "Escaneos" → modo "Activo" → host inyectado → "Resultado del escaneo · Activo" con columna "Componente" al frente, sin tener que ir uno por uno              |
| Playwright — Historial                                                  | Filtro "Activo" nuevo lista los Activos (no Componentes); detalle de un escaneo por Activo hereda la misma columna "Componente" sin código adicional                 |
| Limpieza                                                                | Los activos de prueba (hosts inyectados + par de homónimos) se borraron con el borrado lógico en cascada ya existente (§4.3); confirmado que no queda ninguno activo |

**Tests.** Nuevo test en `ScanServiceTest` ( `start()` para un escaneo de Activo resuelve y setea el `Asset` ) y nuevo test en `VulnerabilityAuditServiceTest` ( `findScanResult()` con `hostAssetId` usa `findByAssetScanWindow`, nunca las otras 3 ramas) — mismo criterio de triangulación ya aplicado al resto de ramas condicionales de ese método. El total de tests backend pasó de 69 a **71** (0 fallos). Sin tests de n8n (sin cambios) ni de frontend (sin test runner configurado, mismo criterio del resto del informe).

## 11.18 Pulido de UX en Centro de Escaneos, a partir de uso real

Tras cerrar el escaneo por Activo (§11.17), el uso en vivo de "Centro de Escaneos" fue señalando una serie de fricciones puntuales de interfaz — cambios exclusivamente de frontend, sin impacto en backend ni en n8n, iterados uno por uno directamente contra la app corriendo.

**Filtros del Historial y del panel "Nuevo escaneo" ocupaban más espacio del necesario.** Ambos bloques ( `card` ) se estiraban al ancho completo del contenedor aunque su contenido (unos pocos campos y dos botones) ocupara mucho menos, dejando una franja vacía a la derecha. Se agregó `w-fit` a ambos `card` para que el fondo se ajuste al contenido, y se pasó de un layout en grid con los botones "Limpiar"/"Buscar" en una fila aparte ( `mt-3` ) a una sola fila ( `flex flex-wrap items-end gap-2` ) con los inputs a ancho fijo ( `w-36 / w-40 / w-48` ) y los botones a continuación de los filtros — no al final de una segunda fila ni empujados a la derecha con `ml-auto`, sino pegados a la izquierda, inmediatamente después de "Objetivo".

**"Nuevo escaneo" con los tabs de modo separados del selector y el botón.** La tira de modos (Componente/Activo/Entorno/Global) estaba en una fila propia arriba, y el `<select>` + "Escanear" en una fila debajo. Se unificó todo en una sola fila ( `flex items-center gap-3` ): tabs de modo, selector del objetivo (ahora a ancho fijo `w-64` en vez de `flex-1 max-w-xs`, que se estiraba de más dentro de un contenedor ya no elástico) y los botones, todos a la par.

**Faltaba un botón "Limpiar" en la pestaña Escaneos.** Historial ya lo tenía; Escaneos no. Se agregó `clearScanFilters()` (resetea la selección de componente/activo/entorno, el resultado del último escaneo y el estado del polling) y su botón correspondiente junto a "Escanear". Al implementarlo se encontró que `useScanPolling().stop()` solo cancelaba el timer interno pero nunca volvía el estado a `'idle'` — sin tocarlo, hacer clic en "Limpiar" en medio de un polling activo habría dejado el botón "Escanear" deshabilitado indefinidamente (la condición `disabled` incluye `poll.state === 'running'`). Se corrigió `stop()` para que también resetee el estado, sin otros llamadores afectados (confirmado por grep — `stop()` no se usaba en ningún otro lugar del proyecto).

**La tabla de resultados no se veía hasta correr un escaneo, y el escaneo sin hallazgos no avisaba nada.** Dos problemas relacionados, pedidos por el usuario en la misma sesión de uso:

1. *Tabla vacía por defecto.* `Table.tsx` cortaba camino cuando `data.length === 0` y devolvía solo un mensaje centrado, sin encabezados — y `GroupedVulnerabilityResult` hacía lo mismo un nivel más arriba antes de llegar a decidir qué columnas armar. Se modificó `Table` para que siempre renderice `<thead>` con las columnas reales y solo reemplace el `<tbody>` por una fila con el mensaje vacío (`colSpan` a todas las columnas) cuando no hay datos; se sacó el corte temprano de `GroupedVulnerabilityResult` para que llegue a construir las columnas correctas (con la columna "Componente" al frente en modo Activo) incluso con 0 filas. El bloque "Resultado del escaneo" de la pestaña Escaneos pasó de mostrarse solo condicionalmente (`resultLoading || scanResult !== null`) a estar siempre visible, con el título ya no dependiente de que hubiera corrido un escaneo (`MODE_LABEL[resultMode ?? scanMode]`), reflejando el modo actualmente seleccionado).
2. *Sin aviso cuando un escaneo termina sin hallazgos.* El toast de finalización (`"Escaneo completo"`) se disparaba siempre igual, sin importar el resultado — un escaneo con 0 vulnerabilidades no se distinguía en nada de uno recién iniciado. Se movió el toast a después de recibir el resultado y se lo condicionó: `toast.success('Escaneo completo')` si hay hallazgos, o un toast informativo (`i`) con el mismo mensaje de `EMPTY_RESULT_MESSAGE[scanMode]` si no los hay. Verificado en vivo disparando un escaneo ACTIVO sobre el componente semilla `PostgreSQL Database / docker` (el caso límite ya conocido de §11.13, sin ecosistema reconocido por GitHub, `scan_id` con 0 filas en `vulnerabilities_audit` confirmado contra la base) — el popup "No se encontraron vulnerabilidades para este componente." apareció apenas terminó el polling.

**El mensaje vacío por defecto era engañoso antes de correr un escaneo.** Al hacer siempre visible la tabla (punto anterior), el mismo texto "No se encontraron vulnerabilidades para este componente." pasó a mostrarse también en el estado inicial, antes de que el usuario disparara nada — dando a entender que ya se había escaneado y no había hallazgos, cuando en realidad no había pasado nada todavía. Se agregó un mensaje distinto para ese caso (`NO_SCAN_YET_MESSAGE`, "Todavía no se disparó ningún escaneo. Elegí un objetivo y presioná 'Escanear' para ver los resultados acá."), elegido en base a si `resultMode` es `null` (nunca corrió un escaneo en esta sesión de la pestaña) o no (un escaneo ya completó, con o sin hallazgos) — `emptyMessage={resultMode ? EMPTY_RESULT_MESSAGE[resultMode] : NO_SCAN_YET_MESSAGE}`.

**Verificación.** Todo confirmado en vivo con Playwright, capturando pantalla en cada paso: tarjetas de filtros ajustadas al contenido en Historial y Escaneos; tabs de modo, selector y botones en una sola fila; botón "Limpiar" nuevo restableciendo el `<select>` a su placeholder; tabla vacía con encabezados visible antes de escanear, con el mensaje de "todavía no se disparó ningún escaneo"; y el popup informativo apareciendo exactamente al completarse un escaneo real sin hallazgos (`SCN-2026-000044`, componente `PostgreSQL Database`).

**Sin tests nuevos.** Cambios exclusivamente de frontend (layout y mensajes), sin test runner configurado para ese lado del proyecto, mismo criterio ya documentado en el resto del informe. El total de tests backend se

mantiene en **71**.

## 11.19 Dashboard: tarjetas de KPI compactas para darle protagonismo a los gráficos

**Problema.** Las 7 tarjetas de KPI (Total Activos, Vulnerabilidades, Abiertas, Críticas, Resueltas este mes, MTTR, Estado del sistema) se mostraban en un grid de 2-3 columnas ( `grid-cols-2 xl:grid-cols-3` ), ocupando 3 filas completas antes de llegar a los gráficos — cada tarjeta bastante grande (ícono de 44px, valor en `text-2xl` , padding `p-5` ). En una pantalla de escritorio típica, eso empujaba "Detectadas vs Resueltas" y "Distribución por Severidad" bien abajo, restándoles protagonismo.

**Solución.** `StatCard.tsx` pasó de un layout horizontal (ícono a la izquierda, texto a la derecha) a uno vertical y compacto: ícono más chico (36px) centrado arriba, título y valor centrados debajo ( `flex flex-col items-center text-center gap-2` , `!p-3` en vez de `p-5` , valor en `text-lg` en vez de `text-2xl` ). El grid de `Dashboard.tsx` pasó de `grid-cols-2 xl:grid-cols-3` a `grid-cols-2 sm:grid-cols-4 lg:grid-cols-7` , poniendo las 7 tarjetas en una sola fila a partir de pantallas medianas-grandes.

**Ajuste encontrado en el camino.** La primera versión (ícono a la izquierda, texto a la derecha, con `truncate` ) cortaba textos largos sin avisar — "Vulnerabilidades" quedaba como "VULNERA..." y el valor de "Estado del sistema" (  `ESTABLE` ) se cortaba directamente a "  E...", perdiendo información real. Se sacó el `truncate` y se dejó que el texto haga wrap normal ( `break-words` , `leading-tight` ) en vez de cortarse — con el layout vertical final el problema dejó de manifestarse en la práctica (todas las 7 tarjetas muestran su texto completo, en una o dos líneas, sin recortes) tanto en 1280px como en 1536px de ancho, verificado con capturas de Playwright en ambas resoluciones.

**Verificación.** Capturas de pantalla con Playwright a 1280px y 1536px de ancho: las 7 tarjetas en una sola fila, sin texto cortado ni desbordado, y los dos gráficos superiores con notoriamente más espacio vertical disponible que antes.

**Sin tests nuevos.** Cambio de frontend puro (layout de un componente ya existente, sin lógica nueva), mismo criterio del resto de esta sección. El total de tests backend se mantiene en **71**.

## Anexo A — Esquema de base de datos final (las 4 migraciones)

---

### Fase 1 — `init/03-software-catalog.sql`

```
CREATE TABLE IF NOT EXISTS public.software_catalog (  
  id          BIGSERIAL PRIMARY KEY,  
  package_name VARCHAR(255) NOT NULL,  
  ecosystem   VARCHAR(50)  NOT NULL,  
  display_name VARCHAR(255),  
  source       VARCHAR(30)  NOT NULL DEFAULT 'GITHUB_ADVISORY',  
  first_seen_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  last_seen_at  TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT unique_catalog_package UNIQUE (package_name, ecosystem)  
);  
  
CREATE EXTENSION IF NOT EXISTS pg_trgm;  
CREATE INDEX IF NOT EXISTS idx_software_catalog_trgm  
  ON public.software_catalog USING gin (package_name gin_trgm_ops);  
  
ALTER TABLE public.assets  
  ADD COLUMN IF NOT EXISTS catalogued BOOLEAN NOT NULL DEFAULT FALSE,  
  ADD COLUMN IF NOT EXISTS pur1      VARCHAR(400);
```

## Fase 2 — `init/04-asset-hierarchy.sql`

```
-- 1) Renombrar la tabla actual (componente de software)
ALTER TABLE public.assets RENAME TO software_components;
ALTER SEQUENCE public.assets_id_seq RENAME TO software_components_id_seq;
ALTER TABLE public.software_components
    RENAME CONSTRAINT unique_asset_per_environment TO unique_component_per_environment;
ALTER TABLE public.software_components
    RENAME CONSTRAINT fk_assets_environment TO fk_components_environment;

-- 2) Nueva tabla assets = el activo real (host / aplicacion / VM / contenedor)
CREATE TABLE IF NOT EXISTS public.assets (
    id          BIGSERIAL PRIMARY KEY,
    name        VARCHAR(150) NOT NULL,
    asset_type   VARCHAR(30)  NOT NULL DEFAULT 'SERVIDOR',
    environment_id BIGINT,
    description  TEXT,
    created_at   TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_assets_environment FOREIGN KEY (environment_id)
        REFERENCES public.environments (id) ON DELETE SET NULL,
    CONSTRAINT unique_asset_name_per_environment UNIQUE (name, environment_id)
);

-- 3) software_components se vincula al nuevo assets
ALTER TABLE public.software_components
    ADD COLUMN IF NOT EXISTS asset_id BIGINT,
    ADD CONSTRAINT fk_components_asset FOREIGN KEY (asset_id)
        REFERENCES public.assets (id) ON DELETE SET NULL;
```

## Fase 3 — `init/05-auth.sql`

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

ALTER TABLE public.users
    ADD COLUMN IF NOT EXISTS password_hash VARCHAR(255);

UPDATE public.users
    SET password_hash = crypt('GefSecure2026!', gen_salt('bf', 10))
    WHERE password_hash IS NULL;

ALTER TABLE public.users
    ALTER COLUMN password_hash SET NOT NULL;
```

## Fase 4 — `init/06-rbac.sql`

```
ALTER TABLE public.users
  ADD CONSTRAINT check_users_role
  CHECK (role IN ('ADMIN','SECURITY_ANALYST','ASSET_OWNER','AUDITOR'));

CREATE TABLE IF NOT EXISTS public.user_asset_assignments (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT NOT NULL REFERENCES public.users(id) ON DELETE CASCADE,
  asset_id    BIGINT NOT NULL REFERENCES public.assets(id) ON DELETE CASCADE,
  assigned_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT unique_user_asset UNIQUE (user_id, asset_id)
);
```

## Mejoras posteriores — `init/08-soft-delete-assets.sql`

```
ALTER TABLE public.assets ADD COLUMN deleted_at TIMESTAMP WITHOUT TIME ZONE;
ALTER TABLE public.software_components ADD COLUMN deleted_at TIMESTAMP WITHOUT TIME ZONE;

-- Backfill: cada componente huerfano se promueve a un activo nuevo con su mismo nombre/entorno
-- (ver detalle completo del loop en §11.3 / archivo real)

ALTER TABLE public.software_components ALTER COLUMN asset_id SET NOT NULL;

ALTER TABLE public.software_components DROP CONSTRAINT fk_components_asset;
ALTER TABLE public.software_components
  ADD CONSTRAINT fk_components_asset FOREIGN KEY (asset_id) REFERENCES public.assets (id);

ALTER TABLE public.assets DROP CONSTRAINT unique_asset_name_per_environment;
CREATE UNIQUE INDEX unique_active_asset_name_per_environment
  ON public.assets (name, environment_id) WHERE deleted_at IS NULL;
```

## Trazabilidad de escaneos — `init/09-scan-traceability.sql`

```
ALTER TABLE public.scan_reports
  ADD COLUMN public_code          VARCHAR(20) UNIQUE,
  ADD COLUMN status               VARCHAR(25) NOT NULL DEFAULT 'COMPLETED',
  ADD COLUMN started_at          TIMESTAMP WITHOUT TIME ZONE,
  ADD COLUMN triggered_by        BIGINT REFERENCES public.users(id),
  ADD COLUMN software_component_id BIGINT REFERENCES public.software_components(id),
  ADD COLUMN environment_id      BIGINT REFERENCES public.environments(id),
  ADD COLUMN error_message       TEXT;

ALTER TABLE public.vulnerabilities_audit
  ADD COLUMN scan_id BIGINT REFERENCES public.scan_reports(id),
  ADD COLUMN asset_vulnerability_id BIGINT; -- FK agregada tras crear la tabla siguiente

CREATE TABLE public.asset_vulnerabilities (
  id BIGSERIAL PRIMARY KEY,
  software_component_id BIGINT NOT NULL REFERENCES public.software_components(id),
  cve_id VARCHAR(50) NOT NULL,
  detection_status VARCHAR(20) NOT NULL DEFAULT 'OPEN',      -- sistema: OPEN/RESOLVED
  triage_status VARCHAR(20) NOT NULL DEFAULT 'DETECTADA',    -- analista: DETECTADA/EN_ANALISIS
  priority VARCHAR(20), cvss VARCHAR(20), exploited BOOLEAN, ghsa_id VARCHAR(30),
  decision TEXT, assigned_to VARCHAR(100) DEFAULT 'SIN_ASIGNAR',
  first_detected_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,
  last_detected_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,
  resolved_at TIMESTAMP WITHOUT TIME ZONE,
  reopen_count INTEGER NOT NULL DEFAULT 0,
  updated_at TIMESTAMP WITHOUT TIME ZONE,
  CONSTRAINT unique_component_cve UNIQUE (software_component_id, cve_id)
);
```

## Catálogos administrables — `init/10-config-catalogs.sql`

```
CREATE TABLE public.asset_types (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(30) UNIQUE NOT NULL
);
INSERT INTO public.asset_types (name) VALUES
  ('SERVIDOR'), ('APLICACION'), ('VM'), ('CONTENEDOR'), ('ENDPOINT');

CREATE TABLE public.ecosystems (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL
);
INSERT INTO public.ecosystems (name) VALUES
  ('npm'), ('pip'), ('maven'), ('nuget'), ('cargo'), ('go'), ('composer'), ('rubygems');
```

## Eliminación del entorno independiente de componente — `init/11-drop-component-`

`environment.sql`

```
-- view_vulnerability_details (init/01-create-tables.sql) dependia de esta columna;
-- no la usa ningun codigo de la aplicacion y ya estaba stale desde la Etapa 3/4 --
-- se dropea en vez de recrearse.
DROP VIEW IF EXISTS public.view_vulnerability_details;

ALTER TABLE public.software_components DROP COLUMN IF EXISTS environment_id;
-- Postgres dropea automaticamente unique_component_per_environment y
-- fk_components_environment junto con la columna.
```

## Anexo B — Archivos principales creados o modificados por fase

**Fase 1:** `init/03-software-catalog.sql` · `model/SoftwareCatalog.java` · `repository/SoftwareCatalogRepository.java` · `service/SoftwareCatalogService.java` · `controller/SoftwareCatalogController.java` · `components/SoftwareAutocomplete.tsx`

**Fase 2:** `init/04-asset-hierarchy.sql` · `model/Asset.java` (nuevo) · `model/SoftwareComponent.java` (ex `Asset.java`) · `controller/AssetController.java` (nuevo) · `controller/SoftwareComponentController.java` (ex `AssetController.java`) · `dto/AssetDTO.java` · `dto/SoftwareComponentDTO.java` · `mapper/AssetMapper.java` · `mapper/SoftwareComponentMapper.java` · `repository/AssetRepository.java` · `repository/SoftwareComponentRepository.java` · `service/AssetService.java` · `service/SoftwareComponentService.java` · `pages/Assets.tsx` (nueva) · `pages/Inventory.tsx`

**Fase 3:** `init/05-auth.sql` · `security/SecurityConfig.java` · `security/JwtService.java` · `security/JwtAuthenticationFilter.java` · `controller/AuthController.java` · `dto/AuthDTO.java` · `exception/InvalidCredentialsException.java` · `contexts/AuthContext.tsx` · `components/RequireAuth.tsx` · `pages/Login.tsx` · `services/api.ts`

**Fase 4:** `init/06-rbac.sql` · `security/AuthenticatedPrincipal.java` · `security/CurrentUser.java` · `model/UserAssetAssignment.java` · `dto/UserAssetAssignmentDTO.java` · `repository/UserAssetAssignmentRepository.java` · `service/UserAssetAssignmentService.java` · `exception/GlobalExceptionHandler.java` (handler de `AccessDeniedException`) · `@PreAuthorize` en `AssetController`, `SoftwareComponentController`, `VulnerabilityAuditController`, `UserController`, `EnvironmentController`, `SystemErrorController`, `ScanController` · `scope-filtering` en `AssetService`, `SoftwareComponentService`, `VulnerabilityAuditService` · `pages/Settings.tsx` (pestaña de asignaciones + gate ADMIN) · `pages/Kanban.tsx` · `pages/Inventory.tsx` · `pages/Assets.tsx` · `pages/Scans.tsx` · `components/Sidebar.tsx` · `components/VulnerabilityCard.tsx` · `types/index.ts` · `services/api.ts` · `vite.config.ts` / `nginx.conf` (fix de colisión de ruta `/assets`)

**Mejoras posteriores (§11):** `repository/VulnerabilityAuditRepository.java` / `service/VulnerabilityAuditService.java` ( `findScanResult` ) · `controller/VulnerabilityAuditController.java` ( `/scan-result` ) · `repository/ScanReportRepository.java` / `controller/ScanController.java` ( `/scan-reports` ) · `pages/Scans.tsx` (reescritura completa) · `init/08-soft-delete-assets.sql` · `model/Asset.java` / `model/SoftwareComponent.java` ( `deletedAt` ) · `repository/AssetRepository.java` / `repository/SoftwareComponentRepository.java` (variantes `...DeletedAtIsNull` / `...DeletedAt` ) ·

`service/AssetService.java` (`delete` en cascada, `restore`, `findDeleted`) ·  
`service/SoftwareComponentService.java` (`delete` lógico, `assetId` obligatorio) ·  
`dto/SoftwareComponentDTO.java` (`assetId` `@NotNull`) · `dto/AssetDTO.java` (`DeletedResponse`) ·  
`controller/AssetController.java` (`/deleted`, `/id/restore`) · `pages/Assets.tsx` (fusión con Inventario +  
 pestañas Vigentes/Eliminados) · `pages/Inventory.tsx` (eliminado) · `App.tsx` / `components/Sidebar.tsx` (ruta y  
 link de Inventario sacados) · `components/Table.tsx` (`onRowClick` / `selectedKey`) · `types/index.ts` /  
`services/api.ts` (tipos y clientes nuevos)

#### **Vulnerabilidades por escaneo, popups y unificación de prioridad (§11.4–§11.6):**

`repository/VulnerabilityAuditRepository.java` (queries nativas  
`findByComponentScanWindow` / `findByEnvironmentScanWindow` / `findByGlobalScanWindow`) ·  
`service/VulnerabilityAuditService.java` (`findByScanReport`) · `repository/EnvironmentRepository.java` /  
`repository/SoftwareComponentRepository.java` (`findFirstByName`) · `repository/ScanReportRepository.java`  
 (`findFirstByTargetTypeAndTargetNameAndExecutedAtLessThan...`) · `controller/ScanController.java` (`/scan-`  
`reports/{id}/vulnerabilities`) · `dto/ScanReportDTO.java` (campo `id`) · `components/Modal.tsx` (tamaño `xl`) ·  
`pages/Scans.tsx` (fila clicable en Historial + popup de detalle) · `pages/Assets.tsx` (popup de software del  
 activo, tamaño `xl`) · `init/02-seed-data.sql` / base de desarrollo (`priority` migrado a inglés) ·  
`types/index.ts` (`VulnPriority` inglés) · `workflows/Pipeline de Gestión Automatizada de Vulnerabilidades`  
 (VMP).json (revisado, sin cambios — ver limitación documentada en §11.6/§10)

#### **Trazabilidad completa de escaneos (§11.8):** `Propuesta_Trazabilidad_Escaneos_GEF_Secure.md` (análisis y

propuesta previos) · `init/09-scan-traceability.sql` · `model/ScanReport.java` (`publicCode`, `status`,  
`startedAt`, `triggeredBy`, `softwareComponent`, `environment`, `errorMessage`, `fix` `@JdbcTypeCode` en  
`environmentBreakdown`) · `model/VulnerabilityAudit.java` (`scanReport`, `assetVulnerability`) ·  
`model/AssetVulnerability.java` (nuevo) · `service/ScanService.java` (nuevo: `start`, `complete`) ·  
`repository/AssetVulnerabilityRepository.java` (nuevo) · `repository/VulnerabilityAuditRepository.java`  
 (`findByScanReport_Id`, `findByAssetVulnerability_Id`) · `service/VulnerabilityAuditService.java`  
 (`applyLifecycle`, redirección de `findAll/findByStatus/findById/updateStatus/delete` a `AssetVulnerability`)  
 · `service/DashboardService.java` (conteos migrados) · `mapper/VulnerabilityAuditMapper.java`  
 (`toResponse` / `applyStatusUpdate` sobrecargados para `AssetVulnerability`) ·  
`controller/WebhookController.java` (`/scan-report` repurpuesto, token interno) ·  
`controller/{SoftwareComponentController,EnvironmentController,ScanController}.java` (disparo crea el  
`Scan` antes de llamar a `n8n`) · `security/SecurityConfig.java` (`permitAll` puntual para el webhook) ·  
`application.properties` / `docker-compose.yml` / `.env.example` (`N8N_INTERNAL_TOKEN`,  
`N8N_BLOCK_ENV_ACCESS_IN_NODE=false`) · `workflows/Pipeline de Gestión Automatizada de Vulnerabilidades`  
 (VMP).json (`Edit Fields`, `Persist_Audit_Findings`, `Persist_Scan_Statistics` → HTTP Request) ·  
`ScanServiceTest.java` (nuevo) · `VulnerabilityAuditServiceTest.java` (tests de ciclo de vida, redirección y  
 comparación entre escaneos)

#### **Comparación entre escaneos (§11.9):** `dto/ScanComparisonDTO.java` (nuevo) ·

`service/VulnerabilityAuditService.java` (`compareScans`, `toMapByCve`) · `controller/ScanController.java`  
 (`/scan-reports/{id}/compare/{otherId}`) · `types/index.ts` (`ScanComparison`, `SeverityChange`) ·  
`services/api.ts` (`scanApi.compare`) · `pages/Scans.tsx` (botón "Comparar escaneos" + popup con selección y  
 las 4 categorías) · `components/Modal.tsx` (`fix: max-h-[90vh]` + scroll interno, header/botón de cerrar siempre  
 visibles) · `dto/ScanReportDTO.java` / `types/index.ts` (campo `publicCode` expuesto, antes solo existía en la  
 base)

#### **Catálogos administrables en Config (§11.11):** `init/10-config-catalogs.sql` (nuevo) ·

`model/AssetType.java` / `model/Ecosystem.java` (nuevos) · `repository/AssetTypeRepository.java` /

`repository/EcosystemRepository.java` (nuevos) · `controller/AssetTypeController.java` / `controller/EcosystemController.java` (nuevos, `/api/asset-types` y `/api/ecosystems`) · `types/index.ts` (`AssetType`, `AssetTypeRequest`, `Ecosystem`, `EcosystemRequest`) · `services/api.ts` (`assetTypeApi`, `ecosystemApi`) · `pages/Settings.tsx` (pestañas Entornos/Tipos de Host/Ecosistemas, con CRUD completo cada una) · `pages/Assets.tsx` (arrays hardcoded de tipo de host y ecosistema reemplazados por `assetTypes` / `ecosystems` cargados de los endpoints nuevos)

**Auditoría de datos congelados vs. en vivo (§11.12):** `init/11-drop-component-environment.sql` (nuevo, incluye `DROP VIEW view_vulnerability_details`) · `model/SoftwareComponent.java` (campo `environment` eliminado) · `dto/SoftwareComponentDTO.java` (`environmentId` eliminado de `Request`) · `mapper/SoftwareComponentMapper.java` (`environmentId` / `environmentName` / `businessCriticality` re-scopeados a `asset.environment.*`) · `service/SoftwareComponentService.java` (`resolveEnvironment` eliminado, chequeo de duplicados re-scopeado vía el activo) · `repository/SoftwareComponentRepository.java` (`findBySoftwareAndVersionAndAsset_Environment_IdAndDeletedAtIsNull`) · `repository/AssetVulnerabilityRepository.java` (`findBySoftwareComponent_Asset_Environment_IdAndDetectionStatus`) · `repository/VulnerabilityAuditRepository.java` (`findByEnvironmentScanWindow`, join agregado contra `assets`) · `service/VulnerabilityAuditService.java` (`findOpenInScope` actualizado; `findByScanReport` prefiere el FK del reporte sobre la búsqueda por nombre) · `mapper/VulnerabilityAuditMapper.java` (`resolveCurrentStatus/Decision/AssignedTo/ResolvedAt`, nuevo) · `types/index.ts` (`environmentId` eliminado de `SoftwareComponentRequest`) · `pages/Assets.tsx` (`<select>` de Entorno del modal de Componente reemplazado por texto de solo lectura heredado del activo) · `VulnerabilityAuditMapperTest.java` (nuevo, primer test de un mapper en el proyecto) · `VulnerabilityAuditServiceTest.java` (tests de resolución por FK vs. por nombre)

**Correlación real de escaneos con GitHub Advisory (§11.13):** `workflows/Pipeline de Gestión Automatizada de Vulnerabilidades (VMP).json` (nodo nuevo `Get_Installed_Packages`; `Fetch_GitHub_Advisories` con `sendQuery` / `queryParameters` `ecosystem + affects` y `onError: continueRegularOutput`; `Check_Internal_Assets` con el JOIN corregido contra `assets`; rewiring de conexiones `Edit Fields` → `Get_Installed_Packages` → `Fetch_GitHub_Advisories`) · `n8n/bootstrap.sh` (defecto identificado en `publish_all()` bajo logging `debug`, no modificado — documentado como hallazgo, no como fix de esta pasada) · `docker-compose.yml` (sin cambios persistentes — `N8N_LOG_LEVEL=debug` usado solo como diagnóstico temporal y revertido)

**Detalle de escaneo por scan\_id real (§11.14):** `service/VulnerabilityAuditService.java` (`findByScanReport` reescrito sobre `findByScanReport_Id`; eliminada la resolución de ventana/activo/entorno) · `repository/EnvironmentRepository.java` (`findFirstByName` eliminado) · `repository/SoftwareComponentRepository.java` (`findFirstByName` eliminado) · `repository/ScanReportRepository.java` (`findFirstByTargetTypeAndTargetNameAndExecutedAtLessThanOrderByExecutedAtDesc` eliminado) · `VulnerabilityAuditServiceTest.java` (5 tests de la heurística reemplazados por 2 directos sobre `scan_id`; 3 tests de `compareScans` actualizados)

**Drill-down por activo en GLOBAL/ENTORNO (§11.15):** `pages/Scans.tsx` (`GroupedVulnerabilityResult`, nuevo componente; reemplaza el renderizado condicional en el resultado en vivo de "Escaneos" y en el popup de detalle del Historial; `Table` reusado sin cambios vía `onRowClick` / `keyField` ya existentes)

*Documento generado como material de referencia técnica para la redacción del informe de tesis — no constituye, por sí mismo, texto final de la tesis.*